



COMSATS University Islamabad, Pakistan

Fake News Detector

By

Jawad Ahmad CIIT/FA22-BSE-015/SWL

Muhammad Abdullah Nasir CIIT/FA22-BSE-022/SWL

Muhammad Ahmad CIIT/FA22-BSE-109/SWL

Supervisor

Ma'am Abida Kausar

Bachelor of Science in Software Engineering (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University Islamabad, Pakistan

Fake News Detector

A project presented to

COMSATS University Islamabad, Sahiwal Campus

In partial fulfilment

of the requirement for the degree of

Bachelor of Science in Software Engineering (2022-2026)

By

Jawad Ahmad CIIT/FA22-BSE-015/SWL

Muhammad Abdullah Nasir CIIT/FA22-BSE-022/SWL

Muhammad Ahmad CIIT/FA22-BSE-109/SWL

DECLARATION

We hereby declare that this Final Year Project titled “**Fake News Detector**” has been carried out by us under the supervision of **Ma’am Abida Kausar**. We further declare that this work is original and has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution. All information derived from published or unpublished sources has been duly acknowledged. If any part of this work is later found to be plagiarized or reproduced without proper citation, we take full responsibility for the consequences.

Jawad Ahmad

M. Abdullah Nasir

M. Ahmad

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) “**Fake News Detector**” was developed by **JAWAD AHMAD (CIIT/FA22-BSE-015/SWL)**, **MUHAMMAD ABDULLAH NASIR (CIIT/FA22-BSE-022/SWL)** and **MUHAMMAD AHMAD (CIIT/FA22-BSE-109/SWL)** under the supervision of Ma’am Abida Kausar and in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

Ma’am Abida Kausar

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr Iftikhar Naseer

Assistant Professor

Department of Computer Science

Superior University Lahore

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

The Fake News Detection System is an intelligent, AI-powered platform designed to help users evaluate the authenticity of news content through advanced natural language processing and verification techniques. In today's digital age, people are constantly exposed to large volumes of information on social media and online platforms, where misinformation and misleading content spread rapidly. However, most individuals lack reliable and accessible tools to verify the credibility of news in real time. This system addresses that challenge by providing a user-friendly mobile solution that analyzes news text using a hybrid deep learning approach based on BERT and a Capsule Network, along with evidence-based verification mechanisms. It enables users to receive instant insights about whether a piece of news is real, fake, or misleading, making it a practical tool for improving information awareness and combating misinformation.

The system uses advanced Machine Learning techniques along with modern NLP frameworks to analyze news text and evaluate its authenticity across three core categories: Real, Fake, and Misleading. The language processing module utilizes BERT to generate deep contextual embeddings, capturing the meaning and intent of the input text. These embeddings are then processed by a Capsule Network, which models hierarchical relationships between words and helps identify complex linguistic patterns often present in misinformation. In addition, the system incorporates an evidence-based verification layer that compares the input with trusted sources to detect contradictions, partial truths, or misleading claims. Together, these components provide users with reliable and interpretable predictions, helping them make informed decisions and reducing the impact of misinformation.

A standout capability of the application is its evidence-based verification layer, which retrieves relevant information from trusted sources to support decision-making. Instead of relying solely on model predictions, the system analyzes external evidence to validate or challenge the given claim. This process includes semantic matching, contradiction detection, and stance analysis to determine whether the information is supported, refuted, or partially accurate. By combining deep learning predictions from BERT and the Capsule Network with real-world evidence, the system provides more reliable and context-aware results, significantly improving its effectiveness in detecting fake and misleading news.

The Fake News Detection System extends beyond basic text classification by integrating into a connected and intelligent verification ecosystem. The platform not only analyzes user-input news but also leverages trusted information sources to provide evidence-based validation of claims. Users can view detailed explanations, including whether a claim is supported, contradicted, or partially accurate based on retrieved content. A dedicated backend system built with secure services such as Firebase manages user data, prediction history, and system interactions, ensuring reliability and scalability. Additionally, an administrative dashboard enables system administrators to monitor user activity, manage datasets, track model performance, and maintain overall system integrity, making the platform robust and adaptable for real-world deployment.

Acknowledgement

All praise is due to Almighty Allah, whose blessings and guidance made it possible for us to complete this project. We express our deepest gratitude to our supervisor, Ma'am Abida Kausar, for his continuous support, valuable feedback, and motivation throughout the project duration.

We would also like to thank our families and friends for their constant encouragement, prayers, and emotional support. Their contributions were essential to the successful completion of this work.

Jawad Ahmad

M. Abdullah Nasir

M. Ahmad

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
API	Application Programming Interface
SRS	Software Requirements Specification
NLP	Natural Language processing
UI	User Interface
UX	User Experience
DB	Database
FR	Functional Requirement
NFR	Non-Functional Requirement
RAM	Random Access Memory
OS	Operating System
IDE	Integrated Development Environment
BERT	Bidirectional Encoder Representations from Transformers
JSON	JavaScript Object Notation
SDK	Software Development Kit
HTTP	Hypertext Transfer Protocol
CapsNet	Capsule Network
LLM	Large Language Model (conceptual reference)
REST	Representational State Transfer
JWT	JSON Web Token

Table of Contents

Chapter 1.....	xiii
1. Introduction.....	1
1.1 Relevance to Course Modules	2
1.2 Project Background.....	4
1.3 Literature Review	5
1.4 Analysis from Literature Review	6
1.5 Methodology and Software Lifecycle	7
1.5.1 Requirement Gathering	7
1.5.2 System Design	8
1.5.3 AI Model Development	9
1.5.4 Implementation	9
1.5.5 Testing and Validation	10
1.5.6 Deployment.....	11
1.5.7 Maintenance	11
Chapter 2.....	61
2. Problem Definition	11
2.1 Problem Statement.....	11
2.1.1 Core Problem Statement:	12
2.2 Deliverables and Development Requirements	12
2.2.1 Major Deliverables	12
2.2.2 Development Requirements	15
2.3 Current System	16
Chapter 3.....	17
3. Requirement Analysis.....	17
3.1 Use Case Diagram	18
3.1.1 Actors	18
3.1.2 Primary Use Cases.....	19
3.2 Use Case Description	20
3.3 Functional Requirements.....	29
3.3.1 User Registration and Login	29
3.3.2 News Content Analysis.....	29
3.3.3 Evidence-Based Verification	29
3.3.4 Prediction Result Display	30

3.3.5 External Evidence Retrieval.....	30
3.3.6 Combined Verification Analysis	30
3.3.7 Prediction History Management.....	30
3.3.8 System Monitoring and Data Management	30
3.3.9 User Interface and Navigation	31
3.3.10 Backend API Functionality.....	31
3.3.11 Data Storage and Management.....	31
3.3.12 System Monitoring and Performance Management	31
3.4 Non-Functional Requirements (NFRs)	31
3.4.1 Usability Requirements.....	31
3.4.2 Performance Requirements.....	32
3.4.3 Scalability Requirements.....	32
3.4.4 Availability Requirements	32
3.4.5 Compatibility Requirements	32
3.4.6 Maintainability Requirements	32
Chapter 4.....	33
4. Design and Architecture	32
4.1 System Architecture.....	32
4.1.1 Presentation Layer (Mobile UI).....	33
4.1.2 Application Layer (Core Logic + AI Processing + Business Logic)	34
4.1.3 Data Layer (Firebase + APIs + Local Storage)	35
4.2 Data Representation	36
4.2.1 Text Input Data	36
4.2.2 AI Model Output Data.....	36
4.2.3 Verification Data	37
4.2.4 Result Data.....	37
4.2.5 User Data (Firebase).....	37
4.2.6 Prediction Data	38
4.2.7 Verification Data	38
4.2.8 System Log Data.....	38
4.2.9 Key Entities.....	38
4.2.10 ER Diagram (Text Representation).....	41
4.3 Process Flow Representation.....	43
4.3.1 General Analysis Workflow (All Modules)	43
4.3.2 Activity Diagrams	45
4.4 Design Models.....	48

4.4.1 Sequence Diagram	48
4.4.2 Class Diagram	49
4.4.3 Architecture Diagram	53
Chapter 5.....	32
5. Implementation	53
5.1 Algorithm	53
5.1.1 User Authentication	53
5.1.2 News Analysis.....	53
5.1.3 Verification and Result Generation.....	54
5.1.4 Prediction History Management.....	54
5.1.5 Store Prediction Result	54
5.1.7 Delete Prediction Record	55
5.1.8 API Request Handling (FastAPI Backend).....	55
5.1.9 System Logging	56
5.2 External APIs	56
5.3 User Interface	57
5.3.1 Sign-up Screen.....	58
5.3.2 Login Screen	59
5.3.3 Home Screen.....	60
5.3.4 Text News Verification	61
5.3.5 Social Media Post Verification	62
5.3.6 Result Screen.....	63
5.3.7 Profile Screen	64
5.3.8 History Screen	65
5.3.9 About Screen.....	66
Chapter 6.....	53
6. Testing and Evaluation	82
6.1 Testing Strategy	82
6.1.1 Unit Testing	82
6.1.2 Integration Testing.....	83
6.1.3 System Testing	84
6.1.4 Performance Testing.....	84
6.1.5 Usability Testing.....	85
6.2 Test Plan.....	86
6.3 Test Environment.....	86
6.3.1 Hardware	86

6.3.2 Software	87
6.4 Test Cases.....	87
6.4.1 News Analysis Test Cases	87
6.4.2 Verification Module Test Cases.....	88
6.4.3 Ambiguous Content Handling	88
6.4.4 Backend API Test Cases.....	89
6.4.5 Authentication Test Cases.....	89
6.4.6 Prediction History Test Cases.....	90
6.4.7 Performance Test Cases.....	90
6.5 Evaluation Results	91
6.5.1 Accuracy Results	91
6.5.2 Performance Results	91
6.5.3 Backend Performance.....	91
6.5.4 Usability Results	91
6.5.5 Robustness.....	92
6.6 Discussion.....	92
Chapter 7.....	81
7. Conclusion and Future Work.....	95
7.1 Conclusion.....	95
7.2 Limitations.....	96
7.2.1 Contextual Ambiguity	96
7.2.2 Misleading Content Complexity.....	96
7.2.3 Dependency on Training Data.....	96
7.2.4 Verification Source Dependence	96
7.2.5 Backend Dependency.....	96
7.3 Future Improvement.....	96
7.3.1 Multilingual Support	97
7.3.2 Improved ML Models	97
7.3.3 Real-Time News Stream Analysis.....	97
7.3.4 Advanced Explainability	97
7.3.5 Scalability Enhancements	97
7.3.6 Browser Extension Integration	97
7.3.7 Continuous Model Updating.....	97
7.4 Final Remarks	98
8. References	99

List of Figures

Figure 3.1: Fake News Detection System Use Case Diagram.....	20
Figure 4.1: Layered Architecture of the Fake News Detection System.....	36
Figure 4.2: ER Diagram.....	42
Figure 4.3: Workflow Diagram.....	44
Figure 4.4: News Analysis Activity Diagram	46
Figure 4.5: Prediction History Diagram	46
Figure 4.6: User Authentication Diagram.....	47
Figure 4.7: Sequence Diagram.....	48
Figure 4.8: Class Diagram (part 1)	49
Figure 4.9: Class Diagram (part 2)	50
Figure 5.0: Class Diagram (part 3)	51
Figure 5.1: Architecture Diagram	54
Figure 5.2: Sign-up Screen	58
Figure 5.3: Login Screen	59
Figure 5.4 Home Screen	60
Figure 5.5: Text News Verification	61
Figure 5.6: Social Media Post Verification	62
Figure 5.7: Result Screen.....	63
Figure 5.8: Profile Screen	64
Figure 5.9: History Screen	65
Figure 6.0: About Screen	66

List of Tables

Table 1.1: Relevance to Course Modules	3
Table 3.1: Use Case 1 – Analyze News Content.....	20
Table 3.2: Use Case 2 - Perform Evidence-Based Verification	21
Table 3.3: Use Case 3 - Display Prediction Results	22
Table 3.4: Use Case 4 - Retrieve External Evidence	23
Table 3.5: Use Case 5 - Manage Prediction History	24
Table 3.6: Use Case 6 - User Authentication & Access	25
Table 3.7: Use Case 7 - System Monitoring & Logging.....	26
Table 3.8: Use Case 8 - Model Update & Performance Monitoring	27
Table 5.1: External APIs and Libraries	56

Chapter 1

INTRODUCTION

1. Introduction

The rapid advancement of digital media and artificial intelligence technologies has significantly transformed how information is created, shared, and consumed. With the widespread use of social media platforms and online news sources, users are constantly exposed to large volumes of information, where misinformation and misleading content can spread rapidly without verification. As a result, individuals often struggle to distinguish between credible and false information. Traditional fact-checking methods rely on manual verification by experts, which is time-consuming and not accessible for everyday users in real time. This creates a critical need for intelligent, automated systems that can assist users in evaluating the authenticity of news content efficiently and reliably.

The Fake News Detection System addresses these challenges through an AI-powered mobile application that performs real-time analysis of news content. Using advanced natural language processing and deep learning techniques, the system evaluates textual information across three key categories: real, fake, and misleading. The core of the system is based on BERT, which generates contextual embeddings to understand the semantic meaning of the input text. These embeddings are then processed by a Capsule Network, which captures hierarchical relationships and complex linguistic patterns often present in misinformation. In addition, the system incorporates an evidence-based verification layer that compares the input with trusted sources to detect contradictions, partial truths, and misleading claims. The model is deployed through an efficient backend and mobile integration, enabling fast inference, low latency, and reliable performance for real-time news verification.

To enhance accuracy and reliability, the system incorporates an evidence-based verification layer that evaluates news claims against trusted sources rather than relying solely on model predictions. This layer performs semantic matching, contradiction detection, and stance analysis to determine whether the information is supported, refuted, or partially accurate. A structured processing pipeline, including text preprocessing, tokenization, and standardized input handling, ensures consistent results across different types of news content. Users can securely authenticate through Firebase, manage their profiles, and store or delete prediction history within a clean and intuitive mobile interface. This integrated approach ensures both robust performance and a seamless user experience for real-time fake news detection.

Beyond individual news verification, the Fake News Detection System functions as a broader multi-role ecosystem that supports users, administrators, and system-level management. Users

can submit news content for analysis, view detailed predictions, and maintain a history of their verification results. The system securely stores user data and prediction logs using Firebase, enabling personalized interaction and tracking. On the administrative side, a dedicated dashboard allows administrators to monitor user activity, manage datasets, analyze model performance, and oversee system operations. This centralized control ensures data integrity, model reliability, and smooth platform functionality, making the system scalable and suitable for real-world deployment.

By combining AI-driven analysis with role-based interaction, the Fake News Detection System creates a unified and intelligent platform that benefits both users and system administrators. It not only enables individuals to verify news content quickly but also allows administrators to manage data, monitor system performance, and maintain model reliability. As a result, the system goes beyond a simple text classification tool and functions as a complete, intelligent news verification ecosystem, helping to reduce the spread of misinformation and improve public awareness.

1.1 Relevance to Course Modules

The development of the Fake News Detection System integrates a wide range of technical concepts, theoretical foundations, and practical skills acquired throughout the Bachelor of Science in Software Engineering (BSSE) program. This project applies knowledge from software engineering, artificial intelligence, natural language processing, mobile and backend development, human-computer interaction, and information security. It demonstrates the combined use of deep learning techniques, including BERT and Capsule Networks, along with API-based system design using FastAPI, Firebase-backed data management, and modern mobile development using Flutter. The system also incorporates role-based interaction, real-time data processing, and optimized UI/UX design, reflecting a comprehensive application of both theoretical and practical computing concepts.

The Fake News Detection System is not limited to a single-user mobile application; it extends into a complete multi-role ecosystem consisting of Users and Administrators, supported by a structured backend and management system. Users interact with the application to verify news content and view prediction results, while administrators oversee system operations, manage datasets, and monitor model performance through a dedicated dashboard. Each course module contributes to different aspects of this system, from AI model development using BERT and

Capsule Networks to frontend design, backend integration, and secure authentication workflows. The relevance of each course module is explained in table 1.1.

Table 1.1: Relevance to Course Modules

Course Module	Application in Fake News Detection System
Software Engineering	Designed complete SRS, system architecture, and UML diagrams (User and Admin flows). Implemented modular design for ML pipeline (BERT + Capsule Network), API structure, and system integration. Developed structured workflows for news input, prediction, and result visualization.
Database Systems	Implemented Firebase Firestore schemas for users, prediction history, and system logs. Used document-based modeling, real-time data handling, and optimized queries for storing and retrieving verification results efficiently.
Artificial Intelligence	Developed hybrid deep learning model using BERT for contextual embeddings and Capsule Network for classification. Performed preprocessing, tokenization, training in two phases, and performance evaluation (accuracy & F1-score).
Natural Language Processing (NLP)	Implemented text preprocessing, tokenization, and contextual understanding of news content. Used WordPiece tokenization and semantic representation to analyze linguistic patterns in fake and misleading news.
Mobile Application Development	Built a cross-platform mobile app using Flutter. Implemented login/signup, news input interface, prediction result screens, history tracking, API integration with FastAPI, and Firebase connectivity.
Human-Computer Interaction (HCI)	Designed a clean and intuitive user interface with proper layout, color coding (Real, Fake, Misleading), readable typography, and smooth navigation. Ensured user-friendly interaction and clear result visualization.
Data Science	Evaluated model performance using accuracy, precision, recall, and F1-score. Analyzed prediction outputs and confidence scores to improve model reliability and interpretability.

Information Security	Implemented secure Firebase Authentication, protected user data, and ensured safe API communication. Applied role-based access control for admin functionalities and secured backend endpoints.
-----------------------------	---

This highlights that the Fake News Detection System is a practical, real-world integration of multiple Computer Science disciplines. The project combines mobile application development, artificial intelligence, natural language processing, human–computer interaction, data science, backend systems, and database management into a unified platform. It fulfills academic objectives while delivering a professional-level news verification system used by end users for real-time analysis and administrators for system monitoring and management.

1.2 Project Background

Traditional news verification relies heavily on manual fact-checking, expert analysis, and dedicated platforms, making real-time validation impractical for everyday users. With advancements in artificial intelligence, natural language processing, and optimized mobile and backend systems, automated fake news detection has become accessible through modern applications, enabling users to verify information quickly and efficiently.

Research indicates that linguistic patterns, writing style, and contextual inconsistencies in text can reveal early signs of misinformation, bias, and misleading claims. Subtle cues such as exaggerated language, lack of credible context, contradictory statements, or emotionally charged phrasing can be effectively captured using natural language processing and deep learning techniques.

The Fake News Detection System leverages these advancements by providing an AI-powered news verification tool that focuses on early awareness of misinformation rather than authoritative fact judgment. The system integrates a hybrid deep learning pipeline based on BERT and a Capsule Network, enabling accurate classification of news content into real, fake, or misleading categories. This approach is further supported by an evidence-based verification layer that analyzes external sources to improve reliability, allowing users to quickly assess the credibility of information in real time.

To enhance contextual accuracy, the system incorporates an evidence-based verification layer that retrieves relevant information from trusted sources and analyzes it alongside the model’s prediction. This allows the system to validate claims through semantic matching, contradiction

detection, and stance analysis, improving the reliability of results beyond standalone classification.

Beyond individual news verification, the system operates within a connected multi-role ecosystem. Users can submit news content, view predictions, and maintain a history of analyzed results, while administrators manage system operations, datasets, and model performance through a centralized dashboard. The platform is supported by a secure backend integrated with Firebase and FastAPI, enabling efficient data handling, real-time communication, and scalable deployment.

The project aligns with global trends in AI-driven information verification by combining natural language processing, deep learning models such as BERT and Capsule Networks, evidence-based validation, role-based interaction, and cloud-backed data management into a unified and intelligent news verification ecosystem.

1.3 Literature Review

Recent research supports the growing use of natural language processing and deep learning for fake news detection and misinformation analysis. Studies such as “Fake News Detection Using Machine Learning” highlight that linguistic pattern, writing style, and contextual inconsistencies can be effectively used to distinguish between real and fake content using large-scale datasets [6] [7]. Similarly, research on “Deep Learning for Text Classification” demonstrates that transformer-based models like BERT can capture contextual meaning, while advanced architectures enhance classification performance by learning deeper semantic relationships within text [4].

Despite recent advancements, existing fake news detection systems remain fragmented and limited in real-world applicability. Many approaches rely solely on text classification models without incorporating external evidence, lack the ability to handle contradictory or misleading information, or depend heavily on static datasets that do not reflect real-time news dynamics. Furthermore, most systems fail to distinguish between verified information and widely circulated misinformation, reducing their reliability in practical scenarios. Research also shows that while AI-based misinformation detection is growing, there is a lack of integrated mobile solutions that combine deep learning models, contextual understanding, and evidence-based

verification into a single unified system capable of providing real-time and reliable news analysis.

Furthermore, current literature does not present a comprehensive news verification ecosystem that effectively combines AI-based predictions with real-time evidence validation and system-level management. Most existing approaches focus on isolated tasks such as text classification or dataset-based evaluation, without integrating trusted-source verification, contradiction analysis, or user interaction within a single platform. There is a lack of research-driven solutions that combine deep learning models like BERT with role-based system architecture, prediction history management, and administrative monitoring. As a result, a complete, unified, and scalable fake news detection ecosystem remains largely unaddressed in current research.

These gaps highlight the need for a holistic platform like the Fake News Detection System, which integrates:

- i. AI-based text classification using deep learning models such as BERT and Capsule Networks,
- ii. automated analysis of news content through natural language processing techniques,
- iii. evidence-based verification using trusted sources with semantic matching, contradiction detection, and stance analysis, and
- iv. a structured, role-based architecture connecting users and administrators within a unified mobile and backend ecosystem.

By addressing both research and practical limitations, the system moves beyond traditional classification tools and positions itself as a next-generation, AI-assisted news verification platform designed to combat misinformation effectively.

1.4 Analysis from Literature Review

The reviewed literature reveals several important insights relevant to the design of a modern news verification system. Most existing fake news detection approaches rely primarily on standalone text classification models, which limits their effectiveness in real-world scenarios where information is dynamic and often contradictory. Research highlights that contextual understanding, linguistic patterns, and semantic relationships play a critical role in identifying misinformation, yet many systems fail to incorporate external evidence for validation.

Furthermore, the integration of advanced deep learning models such as BERT enables deeper contextual analysis of text, improving classification performance and reliability. In addition, combining model predictions with evidence-based verification techniques such as semantic matching, contradiction detection, and stance analysis significantly enhances decision accuracy. These insights reinforce the need for intelligent, automated, and integrated systems that go beyond basic classification to provide reliable and real-time news verification.

Based on these findings, the Fake News Detection System was designed not only as an AI-driven text classification tool but as a complete, integrated verification ecosystem. The platform combines automated news analysis using deep learning models such as BERT and Capsule Networks with an evidence-based verification layer that evaluates claims against trusted sources. By integrating a structured mobile workflow, FastAPI-based backend services, Firebase-backed data storage, and an administrative dashboard for system monitoring and management, the system addresses key limitations identified in the literature while introducing capabilities that extend beyond traditional fake news detection approaches.

1.5 Methodology and Software Lifecycle

The Agile Software Development Lifecycle (SDLC) was adopted for developing the Fake News Detection System. Agile was selected due to its iterative and flexible workflow, which is well-suited for projects involving machine learning models, evolving UI/UX designs, and continuous feature refinement. Since the system includes a hybrid deep learning pipeline (based on BERT and Capsule Network), a verification layer, multiple system roles (User and Admin), and both mobile and backend components, Agile enabled parallel development, rapid improvements, and continuous integration across all modules.

1.5.1 Requirement Gathering

This phase involved identifying and formalizing the needs of all stakeholders, including end users who interact with the application to verify news content and system administrators responsible for managing data, monitoring model performance, and maintaining overall system operations.

The system requirements included defining the complete mobile workflow, starting from Sign Up and Login, followed by user profile creation and navigation to the main application

interface for news verification. The application allows users to input news text, submit it for analysis, and view prediction results along with confidence scores and verification insights.

The project also required selecting suitable machine learning models, including BERT for contextual embedding generation and a Capsule Network for classification. A preprocessing pipeline was designed to handle text cleaning, tokenization, and input formatting compatible with the model.

An important requirement was the integration of an evidence-based verification layer to enhance prediction reliability. This layer performs semantic matching, contradiction detection, and stance analysis using information retrieved from trusted sources. Firebase Authentication was implemented to manage users, supported by a structured Firestore schema designed to store user profiles, prediction history, and system logs.

The system also includes backend services developed using FastAPI to handle API requests, process model inference, and manage communication between the mobile application and the database. For administrative tasks, a dashboard-based management system was required to monitor user activity, manage datasets, and track model performance.

Additionally, role-based access control and Firebase security rules were implemented to ensure secure system usage. Performance considerations were taken into account to ensure efficient model inference and fast response times for real-time news verification. Requirements were iteratively refined throughout the development cycle.

1.5.2 System Design

A complete multi-module architecture was designed for both the mobile application and the backend management system.

The design phase included:

- i. Modular ML pipeline (BERT + Capsule Network)
- ii. Flutter-based mobile app architecture
- iii. Text preprocessing and tokenization workflow
- iv. FastAPI backend architecture for prediction handling
- v. Firestore database schema (users, history, logs)

- vi. Admin dashboard design for system monitoring
- vii. Clean UI wireframes for result visualization
- viii. Data flow diagrams for prediction and storage
- ix. UML diagrams for authentication, prediction flow, and admin operations

This phase ensured a clear structure and communication flow across all system components.

1.5.3 AI Model Development

Three key machine learning components were developed and optimized:

- i. **BERT Model:** Generates contextual embeddings for news text using pre-trained transformer architecture
- ii. **Capsule Network:** Processes BERT embeddings to classify news into Real, Fake, or Misleading
- iii. **Verification Layer:** Performs semantic matching, contradiction detection, and stance analysis using external evidence

The model was trained in two phases: initially freezing BERT to train the Capsule Network, followed by full fine-tuning with a lower learning rate for improved performance.

The system was optimized to achieve low latency during inference and efficient processing for real-time usage. After optimization, the model was validated using both test datasets and real-world news samples to ensure reliable and consistent performance.

This ensured fast and accurate prediction suitable for real-time fake news detection.

1.5.4 Implementation

Implementation occurred in two parallel tracks: Mobile App Development and Backend/API Development.

a) Android App Implementation

The implementation phase included developing a complete Flutter-based interface for news input and result visualization. A structured preprocessing pipeline was implemented to clean and tokenize text before sending it to the backend for analysis. The system integrates API calls

to a FastAPI server, where the BERT + Capsule Network model performs prediction and returns results in real time.

Firebase Authentication was used to manage user access, while Firestore was used to store user profiles and prediction history. The application includes modules for news submission, prediction display (Real, Fake, Misleading), and history management. Role-based access ensures that users and administrators interact with appropriate system features. The app also supports complete prediction management, including saving, viewing, and deleting past verification results, providing a smooth and user-friendly experience.

b) Backend and Admin Management Implementation (FastAPI + Firebase)

The system includes backend-level administrative control to manage system operations and ensure secure functionality. FastAPI was used to handle API requests, model inference, and communication between the mobile application and the database. Secure authentication and role-based access control were implemented using Firebase Authentication to ensure that only authorized users can access system features.

Administrative functionalities include monitoring user activity, managing prediction logs, and maintaining system performance. Firestore was used for real-time data storage and synchronization, allowing efficient handling of user data and prediction history. The backend ensures consistent data flow between all components, enabling reliable and scalable system operation.

Both the mobile application and backend services are integrated with the same Firebase project, ensuring seamless data synchronization and system consistency.

1.5.5 Testing and Validation

The testing phase covered several key areas to ensure the reliability and stability of the Fake News Detection System. Unit testing was conducted to verify the correctness of model inference using the BERT and Capsule Network pipeline. Integration testing validated the complete workflow from user input to preprocessing, API communication, and prediction output. Comprehensive UI and UX testing ensured smooth navigation, clear result visualization (Real, Fake, Misleading), and a consistent user experience across the application. Backend

testing was also performed to verify FastAPI endpoints, JSON request/response handling, and proper integration with Firebase services.

In addition, the evidence-based verification layer was tested to ensure accurate semantic matching, contradiction detection, and stance analysis using retrieved information. Performance testing evaluated response time, system stability, and handling of multiple requests under varying conditions. Finally, model accuracy was validated using both test datasets and real-world news samples to ensure reliable and consistent performance in practical scenarios. Overall, testing ensured the robustness, accuracy, and usability of the system.

1.5.6 Deployment

The deployment phase involved packaging the mobile application for Android devices while optimizing startup time and API response performance for smooth operation. User interface compatibility was ensured across different screen sizes and configurations. Backend services using FastAPI were deployed to handle prediction requests and model inference, with Firebase integrated for secure authentication and real-time data storage. Firestore security rules enforced role-based access control, and basic monitoring was implemented to track system performance. Overall, the system was deployed as a unified architecture ensuring consistency, scalability, and efficient data management.

1.5.7 Maintenance

The maintenance phase involves continuous refinement of the mobile application and backend system to improve usability, performance, and reliability. Ongoing bug fixing is carried out across the application, API services, and machine learning pipeline to ensure stable operation. The deep learning model based on BERT and Capsule Network will be periodically updated to enhance accuracy and adaptability to real-world news data. The evidence-based verification layer will also be improved to provide more reliable validation of claims. Backend services and database structures will be optimized for better performance and scalability, while monitoring mechanisms will be strengthened to track system usage. Future enhancements include expanding support for social media analysis, improving real-time verification capabilities, and integrating more advanced techniques for misinformation detection.

Chapter 2

PROBLEM DEFINITION

2. Problem Definition

Technology has become deeply integrated into everyday life, creating a growing need for automated information verification and intelligent decision-making tools. Although digital platforms provide instant access to vast amounts of information, users still struggle to distinguish between credible and misleading news. Subtle linguistic cues such as exaggerated language, lack of context, or contradictory statements often go unnoticed, allowing misinformation to spread widely. Existing solutions provide limited support, relying heavily on manual fact-checking, offering generic filtering, or lacking advanced AI-based analysis altogether.

At the same time, users seeking reliable information sources or trustworthy verification tools face difficulty in accessing real-time, accurate insights aligned with the content they encounter. Most platforms do not provide contextual validation or explainable results, leading to confusion and misinformation. This gap highlights the absence of a unified system that combines deep learning models like BERT with evidence-based verification and user interaction, enabling intelligent, real-time analysis of news content.

The Fake News Detection System addresses these gaps by providing an AI-powered mobile application that performs automated analysis of news content across three categories: Real, Fake, and Misleading. Supported by a hybrid deep learning pipeline using BERT and a Capsule Network, along with an evidence-based verification layer, the system delivers accurate and context-aware results. It offers a user-friendly interface for real-time news verification, secure Firebase-based data management, and backend support through FastAPI, enabling reliable, scalable, and efficient misinformation detection.

2.1 Problem Statement

Modern digital lifestyles have significantly increased exposure to online information through social media and news platforms, leading to a rapid spread of misinformation and misleading content. Despite the availability of powerful mobile devices and advanced technologies, most users still struggle to verify the authenticity of news in real time. Existing solutions rely heavily on manual fact-checking, provide limited contextual understanding, or fail to incorporate advanced AI-based analysis, leaving users vulnerable to false or biased information.

There is currently no unified, low-cost system that can:

- i. Analyze news content using AI to detect misinformation and misleading patterns
- ii. Provide automated verification without requiring manual fact-checking
- iii. Incorporate contextual understanding through advanced NLP techniques
- iv. Validate claims using evidence-based verification from trusted sources
- v. Support user interaction with features like history tracking and result explanation
- vi. Provide backend-based administrative control for system monitoring and management
- vii. Deliver efficient, real-time performance through optimized mobile and backend integration

As a result, users often rely on unreliable sources, fragmented tools, or personal judgment, while lacking a reliable platform that can intelligently verify news content and present clear, trustworthy insights.

2.1.1 Core Problem Statement:

There is a lack of an accessible, intelligent, and unified system that can analyze news content using AI, incorporate contextual understanding, validate claims through evidence-based verification, and provide real-time insights to help users distinguish between real, fake, and misleading information across mobile platforms.

2.2 Deliverables and Development Requirements

2.2.1 Major Deliverables

2.2.1.1 AI-Powered Fake News Detection Mobile Application

A complete Flutter-based mobile application supporting user interaction for real-time news verification within a unified system. The application allows users to input news content, process it through the AI pipeline, and receive instant predictions along with verification insights.

Hybrid News Analysis System:

- i. **Contextual Understanding Module:** BERT generates deep contextual embeddings from input text

- ii. **Classification Module:** Capsule Network processes embeddings to classify news as Real, Fake, or Misleading
- iii. **Verification Module:** Evidence-based system performs semantic matching, contradiction detection, and stance analysis using trusted sources

All predictions are processed through an optimized backend pipeline to ensure fast response times, reliable performance, and efficient real-time news verification.

2.2.1.2 Evidence-Based Verification Engine

A contextual verification engine that integrates information from trusted sources to enhance prediction accuracy. The system performs semantic matching, contradiction detection, and stance analysis to validate whether a news claim is supported, refuted, or misleading, improving the reliability of real-time news verification.

2.2.1.3 Prediction History Module (Mobile)

A dedicated module allowing users to:

- i. View previously analyzed news inputs
- ii. Access prediction results (Real, Fake, Misleading) with confidence scores
- iii. Review verification details and explanations
- iv. Delete or manage stored prediction history

The module enables users to track past verifications and better understand how news content has been analyzed over time.

2.2.1.4 Result Visualization Module (Mobile)

The system provides a dedicated interface where users can:

- i. View prediction results categorized as Real, Fake, or Misleading
- ii. See confidence scores associated with each prediction
- iii. Access brief explanations or verification insights for the result
- iv. Understand the classification through clear color-coded indicators

Results are presented in a simple and intuitive format, helping users quickly interpret the authenticity of news content.

2.2.1.5 Backend Admin Management (FastAPI + Firebase)

A backend-level management system enabling administrative control over the platform:

- i. Monitor user activity and prediction history
- ii. Manage and update datasets used for model training
- iii. Track system performance and model accuracy
- iv. Maintain overall system integrity through secure access control

This component ensures proper system governance, data reliability, and consistent operation of the fake news detection platform.

2.2.1.6 Backend & Cloud Integration

The Fake News Detection System operates on a shared backend architecture combining FastAPI and Firebase services to support all system components. Firebase Authentication is used to manage secure user access, while Firestore stores essential application data including user profiles, prediction history, and system logs. FastAPI handles API requests, model inference, and communication between the mobile application and the database, ensuring efficient processing of news verification tasks.

This integration ensures secure, scalable, and reliable data management while enabling seamless interaction between the mobile application, machine learning pipeline, and backend services.

2.2.1.7 Complete Technical Documentation

The project documentation includes all essential materials required to understand, develop, and evaluate the Fake News Detection System. It contains the complete Software Requirements Specification (SRS), UML diagrams, and detailed system architecture designs.

The documentation also includes machine learning model development details, training procedures for the BERT and Capsule Network pipeline, and model integration steps. It provides comprehensive information on FastAPI-based backend services and Firebase integration. Additionally, mobile UI/UX flows are documented to illustrate interface design

and navigation logic. Testing reports and performance evaluations are included to demonstrate the system's accuracy, reliability, and overall stability.

2.2.2 Development Requirements

2.2.2.1 Mobile Development

Flutter framework, Dart language, and modern UI components for building a responsive cross-platform mobile application

2.2.2.2 Backend Development

FastAPI for API development, Firebase Authentication for secure user access, and Firestore for database management

2.2.2.3 AI & ML Development

Python, PyTorch / TensorFlow (model training), and integration of BERT with Capsule Network for text classification

2.2.2.4 APIs & Networking

RESTful APIs using FastAPI, HTTP communication for sending user input and receiving predictions, and JSON-based data exchange

2.2.2.5 System Libraries & Dependencies

Transformers library (Hugging Face) for BERT, deep learning libraries for Capsule Network implementation, Flutter packages for UI and API integration, and Firebase SDK (Auth, Firestore)

2.2.2.6 Testing Tools

Android Emulator and physical devices for mobile testing, Postman for API testing, and model evaluation tools for accuracy and performance measurement

2.2.2.7 Management Tools

GitHub for version control and Firebase Console for backend monitoring and data management

2.3 Current System

Current fake news detection solutions provide limited and often surface-level functionality. Many systems rely on basic text classification models or keyword-based filtering, which fail to capture deeper contextual meaning and complex linguistic patterns present in real-world news. Existing approaches often struggle with distinguishing between real, fake, and misleading content, especially when information is partially true or presented in a biased manner. Additionally, most available tools do not incorporate evidence-based verification, making them unreliable when dealing with widely circulated misinformation or contradictory claims.

Many platforms also depend heavily on static datasets and lack real-time adaptability, which reduces their effectiveness in dynamic news environments. Furthermore, existing systems rarely provide user-friendly mobile interfaces for real-time verification, and most do not offer explainable results that help users understand why a piece of news is classified in a certain way. Backend integration, user interaction features, and administrative monitoring are also often missing, limiting the scalability and practical usability of such systems.

The Fake News Detection System addresses these limitations by integrating a hybrid deep learning model based on BERT and a Capsule Network with an evidence-based verification layer. The system supports real-time news analysis through a mobile application, backed by FastAPI and Firebase for efficient data handling and secure user management. By combining contextual understanding, external evidence validation, and role-based system architecture, the platform provides accurate, reliable, and scalable news verification within a unified ecosystem.

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

Requirement analysis defines what the Fake News Detection System must accomplish, how the system should behave, and how its components interact under different operational conditions. Following FYP standards, this chapter includes requirement gathering, use-case modeling, detailed descriptions of workflows, and the specification of both functional and non-functional requirements.

For the Fake News Detection System, requirement analysis focuses on an integrated platform rather than a simple classification tool. The system primarily supports two main actors—User and Admin—each with distinct responsibilities. Users interact with the mobile application to verify news content and view results, while administrators manage system data, monitor performance, and ensure system reliability. Requirement analysis ensures seamless integration between the mobile application, backend services, and machine learning pipeline while maintaining performance, security, and usability.

Core components and modules included in the requirement analysis are:

- i. Text Preprocessing & Tokenization Module (input cleaning and formatting)
- ii. Contextual Embedding Module using BERT
- iii. Classification Module using Capsule Network (Real, Fake, Misleading)
- iv. Evidence-Based Verification Module (semantic matching, contradiction detection, stance analysis)
- v. Prediction Result & Visualization Module (confidence score and explanation)
- vi. User Role: Input news, view results, manage prediction history
- vii. Admin Role: Monitor system usage, manage datasets, track model performance
- viii. Backend Integration Module (FastAPI for API handling and inference)
- ix. Firebase Integration (Authentication and Firestore for data storage)
- x. Real-time processing with optimized performance and low latency
- xi. Clean and user-friendly mobile UI for news verification
- xii. Unified backend architecture ensuring secure and scalable system operation

Overall, requirement analysis ensures that the Fake News Detection System functions as a complete, integrated platform with clearly defined behaviors, data flows, and module interactions, enabling accurate and real-time news verification.

3.1 Use Case Diagram

The use case diagram represents how users interact with the system and how the machine learning pipeline and backend services support these interactions.

3.1.1 Actors

3.1.1.1 User

- i. Inputs news text or social media content for verification
- ii. Views prediction results (Real, Fake, Misleading)
- iii. Accesses confidence scores and verification explanations
- iv. Saves and manages prediction history
- v. Manages user profile

3.1.1.2 Admin

- i. Logs into the system (backend/admin access)
- ii. Monitors user activity and prediction logs
- iii. Manages datasets used for model training
- iv. Tracks system performance and model accuracy
- v. Maintains overall system operations

3.1.1.3 System (AI & Backend Modules)

- i. Text Preprocessing Module (cleaning, tokenization)
- ii. Contextual Embedding Module using BERT
- iii. Classification Module using Capsule Network (Real, Fake, Misleading)
- iv. Evidence-Based Verification Module (semantic matching, contradiction detection, stance analysis)
- v. Handles model inference and prediction generation

3.1.1.4 Backend Services (FastAPI + Firebase)

- i. Handles API requests and responses
- ii. Processes model inference and returns predictions
- iii. Manages user authentication and authorization
- iv. Stores user data and prediction history in Firestore

3.1.2 Primary Use Cases

3.1.2.1 Analyze News Content

User inputs news text, then the system preprocesses and tokenizes the text, then BERT generates contextual embeddings, and then the Capsule Network classifies the news as Real, Fake, or Misleading.

3.1.2.2 Perform Evidence-Based Verification

System retrieves relevant information from trusted sources, then performs semantic matching, contradiction detection, and stance analysis, and then validates whether the claim is supported, refuted, or misleading.

3.1.2.3 Display Prediction Results

System generates prediction output, then calculates confidence scores, and then displays results in a user-friendly format with clear labels (Real, Fake, Misleading).

3.1.2.4 Manage Prediction History

User views previously analyzed news, then accesses stored results and explanations, and then can delete or manage prediction history.

3.1.2.5 User Authentication & Profile Management

User signs up or logs in through Firebase Authentication, then manages profile information, and then securely accesses system features.

3.1.2.6 Backend Processing (API Handling)

System sends user input to FastAPI backend, then processes model inference, and then returns prediction results to the mobile application.

3.1.2.7 Admin System Management

Admin accesses backend system controls, then monitors user activity and prediction logs, then manages datasets and system performance, and then ensures overall system reliability.

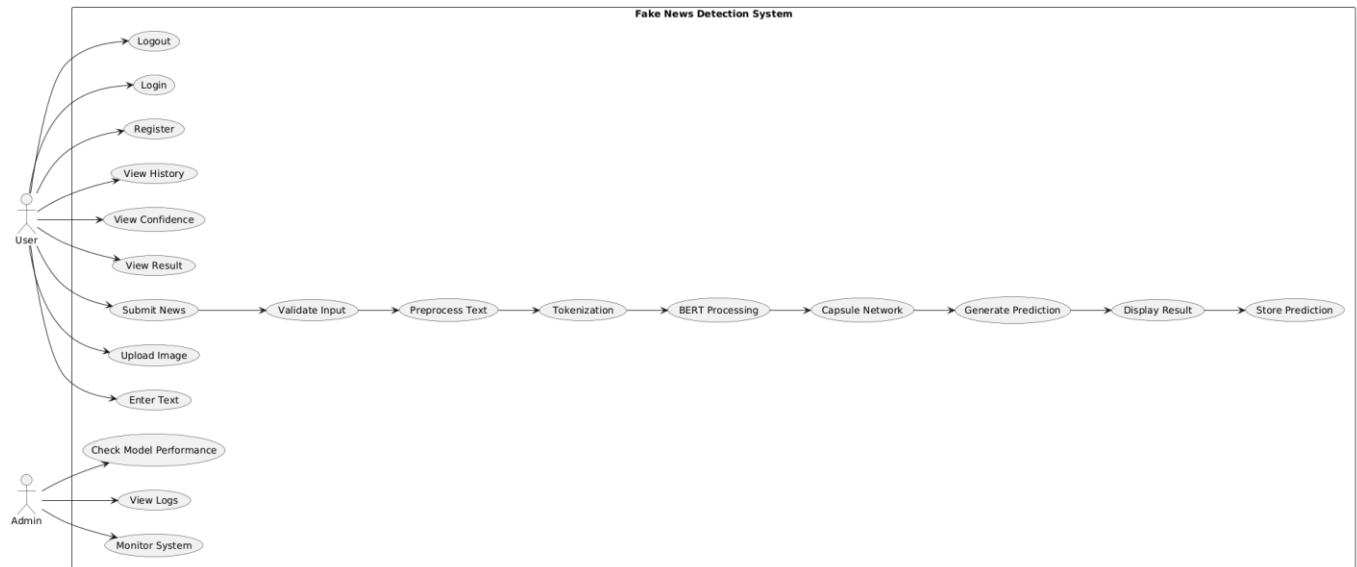


Figure 3.1: Fake News Detection System Use Case Diagram

Figure 3.1 depicts the use case of the system. The system accepts news text input from the user, preprocesses it through tokenization and text cleaning, and then processes it using the BERT and Capsule Network models. The system further performs evidence-based verification using trusted sources before generating the final prediction result.

3.2 Use Case Description

Table 3.1: Use Case 1 – Analyze News Content

Use Case ID	UC-1
Use Case Name	Analyze News Content
Actors	User (Primary), System (Secondary)
Description	User inputs news text. The system preprocesses the text and analyzes it using BERT and a Capsule Network to classify it as Real, Fake, or Misleading, and then provides verification insights.
Trigger	User submits news text for analysis
Preconditions	- User is authenticated - Backend and ML model are available
Post conditions	- Prediction result generated. -Result stored in history. - Output displayed to user

Normal Flow	1. User enters news text. 2. System preprocesses and tokenizes text.3. System generates embeddings using BERT. 4. Capsule Network classifies the input. 5. System displays result (Real, Fake, Misleading).
Alternative Flow	System prompts user to enter valid or complete text if input is insufficient.
Exceptions	Invalid input or server error → display error message.
Business Rules	BR-1: Prediction should be generated within a few seconds for real-time response.

Table 3.1 explains that the system takes news text input from the user and processes it through preprocessing and tokenization before applying the BERT and Capsule Network model. The system then classifies the content as real, fake, or misleading and displays the result along with relevant insights. The process ensures fast, accurate, and real-time analysis of news content.

Table 3.2: Use Case 2 - Perform Evidence-Based Verification

Field	Details
Use Case ID	UC-2
Use Case Name	Perform Evidence-Based Verification
Actors	System (Primary), User (Secondary)
Description	The system retrieves relevant information from trusted sources and analyzes it to validate the input news using semantic matching, contradiction detection, and stance analysis.
Trigger	News content is processed after initial classification
Preconditions	- News text is available - Backend and retrieval system are active
Postconditions	- Verification result generated - Evidence analyzed - Result enhanced with explanation

Normal Flow	1. System receives classified news input. 2. System retrieves relevant articles from trusted sources. 3. Semantic similarity is calculated. 4. System detects contradictions or support. 5. Final verification result is updated.
Alternative Flow	If no relevant sources are found, system marks the result based on model prediction only.
Exceptions	API failure or no data retrieved → system proceeds without verification layer.
Business Rules	BR-1: Only trusted sources should be used for verification. BR-2: Verification must not significantly delay response time. BR-3: Results must be explainable to the user.

Table 3.2 explains that the system enhances prediction accuracy by retrieving information from trusted sources and analyzing it through semantic matching and contradiction detection. This process helps determine whether a claim is supported or refuted, improving the reliability of the result. The system ensures efficient and real-time verification while maintaining accuracy and interpretability.

Table 3.3: Use Case 3 - Display Prediction Results

Field	Details
Use Case ID	UC-3
Use Case Name	Display Prediction Results
Actors	User (Primary), System (Secondary)
Description	The system presents the analyzed news result to the user, including classification (Real, Fake, Misleading), confidence score, and verification insights.
Trigger	News analysis and verification process is completed
Preconditions	- News content has been processed - Prediction result is available
Postconditions	- Result displayed to user - Prediction stored in history

Normal Flow	1. System receives final prediction. 2. System prepares result output. 3. Classification label is displayed (Real/Fake/Misleading). 4. Confidence score is shown. 5. Verification explanation is provided.
Alternative Flow	If confidence is low, system highlights uncertainty in result.
Exceptions	Error in prediction or API failure → display error message.
Business Rules	BR-1: Results must be displayed clearly and quickly. BR-2: Output must be easy to understand for users. BR-3: Prediction results must be stored securely.

Table 3.3 explains that the system displays the analyzed news result to the user in a clear and user-friendly format. It shows whether the news is real, fake, or misleading, along with a confidence score and relevant verification insights. The process ensures fast, understandable, and reliable communication of results to the user.

Table 3.4: Use Case 4 - Retrieve External Evidence

Field	Details
Use Case ID	UC-4
Use Case Name	Retrieve External Evidence
Actors	System (Primary), External News Sources, User
Description	The system retrieves relevant information from trusted sources to support verification of the input news.
Trigger	Verification process is initiated after news analysis
Preconditions	- Internet connection available - External sources accessible - Backend services active
Postconditions	- Relevant articles retrieved - Evidence processed for verification
Normal Flow	1. System sends request to retrieve related news/articles. 2. External sources return data. 3. System validates and filters relevant content. 4.

	Evidence is processed for semantic matching. 5. Verification module uses this data to refine prediction.
Alternative Flow	If sources are unavailable, system proceeds using model prediction only.
Exceptions	API failure or no data retrieved → continue without evidence layer.
Business Rules	BR-1: Only trusted sources should be used. BR-2: Data must be validated before use. BR-3: Retrieval should not delay response significantly. BR-4: Maintain system performance and reliability.

Table 3.4 explains that the system retrieves relevant information from trusted sources to support the verification process. This external evidence is analyzed through semantic matching and contradiction detection to improve prediction accuracy. If external data is unavailable, the system continues with the model's prediction, ensuring uninterrupted operation while maintaining efficiency and reliability.

Table 3.5: Use Case 5 - Manage Prediction History

Field	Details
Use Case ID	UC-5
Use Case Name	Manage Prediction History
Actors	User (Primary), System (Secondary)
Description	The system stores analyzed news results and allows users to view, manage, or delete their prediction history.
Trigger	User accesses history section or completes a new prediction
Preconditions	- User is authenticated - At least one prediction exists
Postconditions	- Prediction history displayed - Data updated or deleted as requested
Normal Flow	

	1. System stores prediction after analysis. 2. User opens history module.3. System retrieves stored predictions from database. 4. User views results and details. 5. User deletes or manages selected entries.
Alternative Flow	If no history exists, system displays an empty state message.
Exceptions	Database error → show retrieval failure message.
Business Rules	BR-1: All user data must be securely stored. BR-2: Only the authenticated user can access their history. BR-3: History operations must be efficient and responsive

Table 3.5 explains that the system stores prediction results and allows users to manage their verification history. Users can view past analyses, understand results, and delete records if needed. This feature enhances usability and provides users with a structured way to track their previous interactions with the system.

Table 3.6: Use Case 6 - User Authentication & Access

Field	Details
Use Case ID	UC-6
Use Case Name	User Authentication & Access
Actors	User (Primary), System (Secondary)
Description	Users create accounts, log in securely, and gain access to the fake news detection system features.
Trigger	User opens the application and selects login or signup
Preconditions	- Internet connection available - Firebase Authentication configured
Postconditions	- User authenticated - Access granted to system features
Normal Flow	1. User enters login/signup details. 2. System verifies credentials using Firebase Authentication. 3. Upon success, user is granted access to the

	application. 4. User proceeds to use system features.
Alternative Flow	If credentials are incorrect, system prompts user to retry or reset password.
Exceptions	Authentication failure or network error → display error message.
Business Rules	BR-1: User data must be securely stored and encrypted. BR-2: Only authenticated users can access system features. BR-3: Session management must ensure secure access.

Table 3.6 explains that the system provides secure user authentication to control access to the application. Users can sign up, log in, and access features such as news verification and history management. This ensures data security, user privacy, and controlled system usage.

Table 3.7: Use Case 7 - System Monitoring & Logging

Field	Content
Use Case ID	UC-7
Use Case Name	System Monitoring & Logging
Actors	System (Primary)
Description	The system automatically monitors user activity, API requests, and prediction operations by maintaining logs for performance tracking and debugging.
Trigger	Any user interaction such as login, news analysis, or prediction storage
Preconditions	- Backend services are active - Logging mechanism initialized
Post conditions	- Logs are recorded successfully - System performance data available
Normal Flow	User performs an action (login, analysis, etc.)
Alternative Flow	If logging fails, system continues operation without interrupting the user flow.

Field	Content
Exceptions	Logging service unavailable → system stores minimal fallback logs or skips logging.
Business Rules	BR-1: Logs must not expose sensitive user data. BR-2: Logging should not affect system performance. BR-3: Logs should be maintained for debugging and performance analysis.

Table 3.7 explains that the system automatically tracks user actions, prediction processes, and API activity through internal logging mechanisms. These logs help monitor system performance, detect errors, and support debugging. This functionality ensures system reliability, security, and continuous improvement without requiring a dedicated admin module.

Table 3.8: Use Case 8 - Model Update & Performance Monitoring

Field	Content
Use Case ID	UC-8
Use Case Name	Model Update & Performance Monitoring
Actors	System (Primary)
Description	The system manages model performance by monitoring prediction accuracy and updating models when improved versions are available.
Trigger	Periodic evaluation or deployment of updated model
Preconditions	- Backend system active - Model files available
Post conditions	- Model updated successfully - Performance metrics refreshed
Normal Flow	1. System evaluates model performance using recent prediction data. 2. New or improved model version is loaded. 3. System replaces or updates existing model. 4. Updated model is used for future predictions.

Field	Content
Alternative Flow	If no improved model is available, system continues using the current model
Exceptions	Model update failure → system reverts to previous stable model.
Business Rules	BR-1: Model updates must not interrupt user operations. BR-2: System must maintain prediction consistency after updates. BR-3: Model versions should be tracked for performance comparison.

Table 3.8 explains that the system automatically monitors model performance and updates machine learning models when improved versions are available. This ensures that the fake news detection system remains accurate, efficient, and adaptable to new patterns in data without requiring manual intervention.

Table 3.9: Use Case 9 - Combine Prediction with Evidence Verification

Field	Content
Use Case ID	UC-9
Use Case Name	Combine Prediction with Evidence Verification
Actors	System (Primary)
Description	The system combines model predictions with evidence retrieved from trusted sources to refine the final classification of news content.
Trigger	Initial prediction generated and external evidence available
Preconditions	- Prediction result available - External evidence retrieved or accessible
Post conditions	- Final classification updated - Explanation generated and displayed
Normal Flow	1. System collects prediction from BERT + Capsule Network. 2. Retrieves relevant articles from trusted sources. 3. Applies semantic matching and contradiction detection. 4. Combines model output with evidence insights. 5. Generates result and displays to user.

Field	Content
Alternative Flow	If external evidence is unavailable, system uses model prediction only.
Exceptions	Conflicting evidence → system marks result as potentially misleading and logs inconsistency.
Business Rules	BR-1: Trusted sources must be prioritized. BR-2: Evidence must be validated before use. BR-3: System must provide explanation with final result.

Table 3.9 explains that the system enhances prediction accuracy by combining deep learning model outputs with evidence from trusted sources. This process ensures that the final result is more reliable and context-aware, improving the system’s ability to handle real-world misinformation scenarios.

3.3 Functional Requirements

3.3.1 User Registration and Login

The system must provide secure account creation and login using Firebase Authentication. Users register with email and password, and the system validates credentials during login. All authentication processes must prevent unauthorized access, maintain active sessions, and securely store user-related data, including prediction history.

3.3.2 News Content Analysis

The system must analyze user-input news text using BERT and a Capsule Network. The module must preprocess and tokenize the text, generate contextual embeddings, and classify the content as Real, Fake, or Misleading. Results must be generated quickly and displayed clearly to the user.

3.3.3 Evidence-Based Verification

The system must validate analyzed news by retrieving relevant information from trusted sources. It must perform semantic matching, contradiction detection, and stance analysis to

enhance prediction accuracy. The system must provide verification insights alongside the classification result, ensuring reliability and interpretability of the output.

3.3.4 Prediction Result Display

The system must display the classification result of the analyzed news as Real, Fake, or Misleading along with a confidence score. The results must be presented in a clear, user-friendly format with visual indicators and brief explanations to help users understand the outcome.

3.3.5 External Evidence Retrieval

The system must retrieve relevant information from trusted sources to support verification of the input news. If external data is unavailable, the system must continue using the model prediction without interrupting functionality.

3.3.6 Combined Verification Analysis

The system must generate a result by combining deep learning predictions with evidence-based verification. It must perform semantic matching, contradiction detection, and stance analysis to improve accuracy and provide reliable insights.

3.3.7 Prediction History Management

The system must allow users to save and view previously analyzed news results. Each saved record must include the input text, prediction result, and timestamp. Users must also be able to delete stored history, and all data must be securely maintained.

3.3.8 System Monitoring and Data Management

The system must provide administrative functionality to monitor user activity, manage datasets, and track model performance. Access to these features must be restricted to authorized administrators, ensuring secure and controlled system operation.

3.3.9 User Interface and Navigation

The system must provide a clean and user-friendly mobile interface that allows easy access to news input, prediction results, history, and profile settings. Navigation must be smooth, optimized for mobile devices, and designed to minimize steps required for verifying news content.

3.3.10 Backend API Functionality

The system must provide backend services using FastAPI to handle user requests, process model inference, and return prediction results. API communication must be efficient, secure, and capable of handling multiple requests with minimal latency.

3.3.11 Data Storage and Management

The system must store user data and prediction history securely using Firebase Firestore. It must allow users to retrieve, manage, and delete stored records while ensuring data consistency and integrity.

3.3.12 System Monitoring and Performance Management

The system must provide internal monitoring capabilities to track user activity, API requests, and overall system performance. Logging mechanisms should record important events such as authentication, prediction processing, and database operations. All system activities must be handled securely, ensuring that sensitive user data is protected. Monitoring and logging should support error detection, performance optimization, and system reliability without requiring manual administrative control.

3.4 Non-Functional Requirements (NFRs)

3.4.1 Usability Requirements

The application must provide an intuitive and user-friendly interface that allows users to verify news in a few simple steps. Buttons, labels, and instructions must be clear and consistent, and results must be displayed in an easy-to-understand format with proper visual hierarchy.

3.4.2 Performance Requirements

The system must deliver fast performance suitable for real-time news verification. At least 95% of predictions must be generated within a few seconds. The deep learning pipeline using BERT and Capsule Network, along with backend processing, must be optimized for efficient response time.

3.4.3 Scalability Requirements

The system architecture must be modular to support future enhancements such as social media analysis, multilingual support, and improved verification mechanisms. It should be capable of handling many users and requests without requiring major system redesign.

3.4.4 Availability Requirements

The system must maintain high availability for backend services and API communication. It should handle network interruptions gracefully and ensure continuous operation by providing fallback mechanisms when external verification sources are unavailable.

3.4.5 Compatibility Requirements

The application must support a wide range of Android devices and operate across different screen sizes and hardware configurations. All features, including API communication and result display, must function reliably on supported devices.

3.4.6 Maintainability Requirements

The system must be maintainable through modular code design. Machine learning models and backend components should support periodic updates and improvements. Logging mechanisms must capture errors and system activity to facilitate debugging and future enhancements.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

The design and architecture define how the Fake News Detection System operates internally, how its components interact, and how data flows between the mobile application and backend services. The system is built using a modular, layered architecture that supports real-time news analysis, deep learning-based classification, evidence-based verification, and centralized data management. This architecture ensures scalability, maintainability, and consistent performance across all system components.

The mobile application is designed to provide a simple and intuitive interface for users to input news content, view prediction results, and manage their verification history. The system focuses on a user-centered workflow, where users interact with the application to analyze news and receive instant feedback. Administrative functionality is handled through backend controls, enabling monitoring of user activity, dataset management, and system performance tracking.

On the technical side, the system integrates a hybrid deep learning pipeline using BERT and a Capsule Network for text classification. FastAPI is used for backend services to handle API requests and model inference, while Firebase Authentication ensures secure user login and Firestore serves as the central database for storing user data and prediction history. Additionally, the system incorporates an evidence-based verification layer that retrieves information from trusted sources to enhance prediction accuracy. These components collectively enable accurate, real-time, and reliable news verification while maintaining a clean and user-friendly mobile interface.

4.1 System Architecture

The Fake News Detection System adopts a modular three-tier architecture designed to support real-time news analysis, efficient backend processing, and secure data management. The architecture consists of three main layers:

- **Presentation Layer (Mobile App):** Handles user interaction, news input, and result visualization.
- **Application Layer (Backend – FastAPI):** Processes requests, performs model inference, and manages system logic.
- **Data Layer (Firestore):** Stores user information, prediction history, and system logs.

This architecture ensures scalability, maintainability, and high performance across the mobile and backend environments, enabling a reliable and efficient fake news detection platform.

4.1.1 Presentation Layer (Mobile UI)

This layer handles all user-facing interactions within the mobile application.

4.1.1.1 Mobile Application (Flutter)

The application is used by end users to input news content, view prediction results, and manage their verification history.

4.1.1.2 User Interface

The user interface of the application provides a simple and structured workflow for news verification. Users can enter or paste news text into the input field and submit it for analysis. The system then displays AI-generated results, including classification labels (Real, Fake, Misleading), confidence scores, and verification insights.

A dedicated result panel presents the outcome in a clear and visually understandable format using color-coded indicators. The interface supports smooth navigation between key sections such as Home, Verify News, History, and Profile. Users can also view their previously analyzed news and delete records as needed, ensuring full control over their prediction history. The design emphasizes clarity, minimal interaction steps, and ease of use.

4.1.1.3 Admin Interaction (Backend Access)

- i. Monitor user activity and prediction logs
- ii. Manage datasets and system configurations
- iii. Track model performance and system behavior
- iv. Ensure secure and reliable system operation

4.1.1.4 Admin Control (Backend + Firebase)

- i. Secure login for admin through backend authentication
- ii. Monitoring of user activity and prediction history
- iii. Management of datasets and system configurations
- iv. Tracking system performance and model accuracy
- v. Real-time data updates using Firebase Firestore

4.1.2 Application Layer (Core Logic + AI Processing + Business Logic)

This layer represents the intelligence of the system and handles all processing, model inference, and decision-making.

4.1.2.1 AI Processing

Text preprocessing including cleaning, normalization, and tokenization of user input.

Deep learning pipeline for analysis:

- i. Contextual embedding using BERT
- ii. Capsule Network for classification (Real, Fake, Misleading)
- iii. Confidence scoring and probability estimation
- iv. Interpretation of model outputs into readable results

4.1.2.2 Evidence-Based Verification Layer

The system incorporates an evidence-based verification layer that retrieves relevant information from trusted sources to enhance prediction accuracy. It performs semantic similarity matching to identify related content, detects contradictions between the input news and external evidence, and applies stance analysis to determine whether the evidence supports, refutes, or remains neutral toward the claim. These insights are then combined with the model's prediction to produce a more reliable and context-aware final decision.

4.1.2.3 Business Logic

- i. News input processing and prediction generation
- ii. API request handling using FastAPI
- iii. Prediction history management (save, retrieve, delete)
- iv. User authentication and session handling
- v. Role-based access control (User and Admin)
- vi. Error handling and response management

4.1.2.4 Result Generation Engine

Merges:

- i. Model predictions (BERT + Capsule Network)
- ii. Evidence-based verification outputs

- iii. Confidence scores

Produces:

- i. Final classification (Real, Fake, Misleading)
- ii. Confidence level
- iii. Verification explanation and insights

4.1.3 Data Layer (Firebase + APIs + Local Storage)

This layer handles data storage, retrieval, and communication between system components.

4.1.3.1 Local Storage

Temporary storage of user input and processed data during analysis + Caching of recent prediction results for faster access

4.1.3.2 External APIs

- i. Retrieval of relevant information from trusted news sources for verification
- ii. Supports evidence-based analysis to enhance prediction accuracy

4.1.3.3 FastAPI Backend

The FastAPI backend handles API requests from the mobile application, performs text preprocessing and tokenization, and runs model inference using BERT and the Capsule Network. It also communicates with Firebase to store and retrieve user data and prediction history.

4.1.3.4 Firebase Backend (Authentication + Firestore)

Stores:

- i. Manages secure user authentication
- ii. Stores user profiles and prediction history
- iii. Ensures real-time data synchronization

4.1.3.5 Session Data

- i. Temporary values during API requests and processing
- ii. Cached results to reduce repeated computations and improve performance

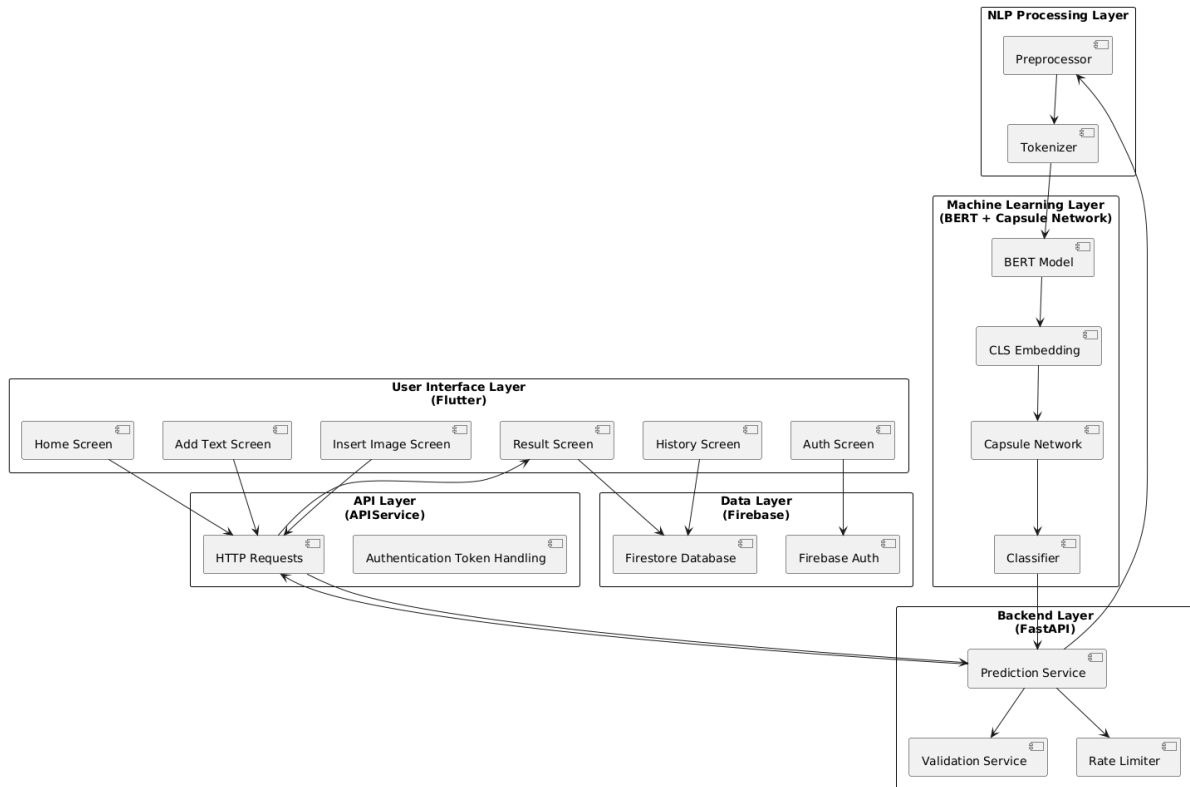


Figure 4.1: Layered Architecture of the Fake News Detection System

Figure 4.1 show the overall system architecture.

4.2 Data Representation

This section represents the internal data structure of the system.

4.2.1 Text Input Data

- i. News text entered or pasted by the user
- ii. Preprocessed through cleaning and normalization
- iii. Tokenized into sub-word units
- iv. Converted into numerical embeddings for model input

4.2.2 AI Model Output Data

- i. Classification result: Real, Fake, or Misleading

- ii. Confidence score for prediction
- iii. Contextual embedding output from BERT

Hence, confidence values and predicted labels are generated and passed to the result module.

4.2.3 Verification Data

- i. Retrieved content from trusted sources
- ii. Semantic similarity scores
- iii. Contradiction detection results
- iv. Stance classification (Support / Refute / Neutral)

This data is used to refine and validate the final prediction.

4.2.4 Result Data

Generated by combining model predictions and verification outputs:

- i. Final classification (Real, Fake, Misleading)
- ii. Confidence score
- iii. Explanation or verification insights

4.2.5 User Data (Firebase)

Stored in Firebase:

4.2.5.1 User

- i. User ID (UID), name, and basic profile information
- ii. Email and authentication details
- iii. Prediction history
- iv. Timestamps of analysis
- v. User preferences/settings

4.2.5.2 Admin Data

- i. Admin ID
- ii. Access credentials
- iii. System logs and monitoring data

- iv. Dataset management records
- v. Model performance metrics

4.2.6 Prediction Data

Each prediction record includes:

- i. Input news text
- ii. Predicted label (Real / Fake / Misleading)
- iii. Confidence score
- iv. Model output probabilities
- v. Timestamp
- vi. User ID

4.2.7 Verification Data

Contains:

- i. Retrieved articles or evidence from trusted sources
- ii. Semantic similarity scores
- iii. Contradiction detection results
- iv. Stance classification (Support / Refute / Neutral)
- v. Source references

4.2.8 System Log Data

Used for:

- i. Tracking user activity
- ii. Monitoring API requests and responses
- iii. Recording errors and exceptions
- iv. Performance analysis and system monitoring.

4.2.9 Key Entities

4.2.9.1 User

Represents a logged-in user authenticated via Firebase.

Stores:

- i. Login/session state
- ii. Profile information (name, email)
- iii. Prediction history
- iv. Timestamps of analysis
- v. User preferences/settings

4.2.9.2 TextData

Contains:

- i. Raw news text entered by the user
- ii. Pre-processed text (cleaned and normalized)
- iii. Tokenized representation
- iv. Encoded input for model inference

4.2.9.3 AIModel

Includes:

- i. Model configuration for BERT
- ii. Capsule Network architecture details
- iii. Input formatting and tokenization parameters
- iv. Model version and training metadata

4.2.9.4 VerificationInfo

Stores:

Temperature, Humidity, Timestamp of last refresh and Cached values used when API is unavailable

4.2.9.5 AnalysisResult

Contains:

- i. Retrieved data from trusted sources
- ii. Semantic similarity scores

- iii. Contradiction detection results
- iv. Stance classification (Support / Refute / Neutral)
- v. Source references

4.2.9.6 ResultExplanation

Stores final explanation including:

- i. Summary of prediction reasoning
- ii. Evidence-based insights
- iii. Confidence interpretation
- iv. Highlighted decision factors

4.2.9.7 PredictionRecord

Represents a saved prediction. Stores:

- i. Input news text
- ii. Prediction result
- iii. Confidence score
- iv. Verification insights
- v. Timestamp
- vi. Linked user ID

4.2.9.8 SystemLog

Represents a clinic account authenticated via Firebase. Stores:

- i. API request and response logs
- ii. Error and exception records
- iii. System performance data
- iv. User activity tracking

4.2.9.9 Admin

Represents the system administrator. Stores:

- i. Admin authentication details
- ii. Dataset management records

- iii. System monitoring logs
- iv. Model performance tracking.

4.2.9.10 Prediction Record

Represents a saved news verification result. Stores:

- i. Input news text
- ii. Predicted class (Real / Fake / Misleading)
- iii. Confidence score
- iv. Verification insights (evidence summary)
- v. Timestamp
- vi. Linked user ID

4.2.10 ER Diagram (Text Representation)

Since graphical rendering is limited, the ER diagram is presented as a structured textual representation. The system is composed of several core entities, including User, Text_Data, Prediction_Record, Analysis_Result, and Verification_Info, where each entity represents a specific functional component of the Fake News Detection workflow. The User entity stores user-related information such as username, email, password, and account details. A single user can perform multiple news verification operations, creating a one-to-many relationship between the User and Prediction_Record entities.

The Prediction_Record entity maintains the complete history of predictions generated by the system. It stores details such as prediction ID, prediction date and time, confidence score, and the final classification result. Each prediction record is connected to a specific Text_Data entity, which contains the actual news content entered by the user, including title, article text, or extracted social media post text. This ensures that both the original input and its processed form are preserved for future reference and analysis.

The Analysis_Result entity stores the output generated by the BERT and Capsule Network model. It includes the predicted class (Real or Fake), confidence percentage, and additional explanation generated by the verification module. This entity is directly associated with the Prediction_Record entity to maintain a structured relationship between user inputs and model outputs.

Additionally, the Verification_Info entity contains supporting information related to the prediction process, such as source references, related article links, and verification explanations used to justify the final prediction. This entity improves transparency and helps users understand why a specific news item was classified as real or fake. Together, these entities create a well-structured database design that supports efficient storage, retrieval, tracking, and analysis of fake news verification activities within the application.

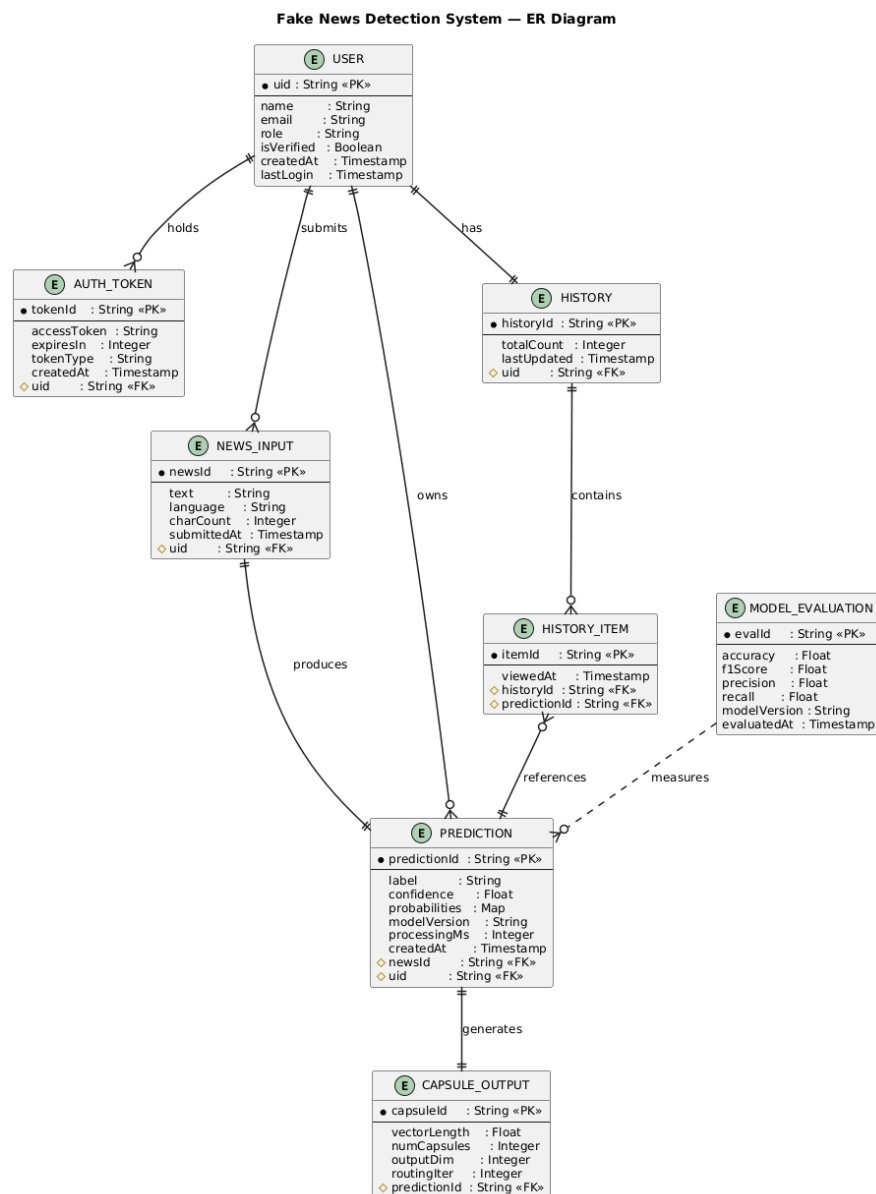


Figure 4.2: ER Diagram

Figure 4.2 shows how the Fake News Detection System stores and links users, prediction records, text data, and model outputs. Each prediction record is associated with the user's input text and its corresponding analysis results, including classification labels and confidence scores. Verification data from trusted sources is also linked to enhance result accuracy and provide explanations. Overall, the structure ensures that data remains organized, easily accessible, and traceable across all system components.

4.3 Process Flow Representation

The Fake News Detection System follows a structured workflow to analyze news content, verify its authenticity, and present the results to the user. The process begins when the user inputs news text through the application interface, which is then transmitted to the backend for processing. The system performs text preprocessing, including cleaning and tokenization, before generating contextual embeddings using the BERT model. These embeddings are classified using a Capsule Network to determine whether the news is Real, Fake, or Misleading. Additionally, a verification layer retrieves supporting evidence from trusted sources to enhance the reliability of the prediction. The final output, including the prediction label, confidence score, and explanation, is displayed to the user and can be stored for future reference.

4.3.1 General Analysis Workflow (All Modules)

The system follows a sequential workflow to process and analyze news text efficiently. Initially, the user inputs news content through the application interface. The input text is then transmitted to the FastAPI backend, where preprocessing steps such as cleaning, normalization, and tokenization are performed.

After preprocessing, the text is passed to the BERT model, which generates contextual embeddings representing the semantic meaning of the input. These embeddings are then forwarded to the Capsule Network, which performs classification and predicts whether the news is Real, Fake, or Misleading, along with a confidence score.



Figure 4.3: Workflow Diagram

Figure 4.3 outlines the complete operation of the system, starting from user login to news input, text preprocessing, and model inference. The input text is processed through the BERT and Capsule Network models, while evidence retrieval from trusted sources runs in parallel for verification. The outputs from both the model and verification layer are combined to generate the final prediction with confidence and explanation. The process concludes by displaying the result to the user and providing options to save or delete it from the prediction history. It represents the complete processing pipeline behind each news analysis.

4.3.2 Activity Diagrams

4.3.2.1 News Analysis Activity

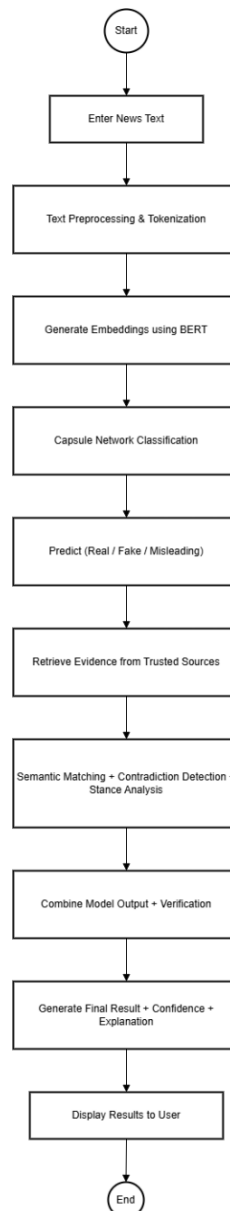


Figure 4.4: News Analysis Activity Diagram

Figure 4.4 outlines that the news analysis process begins with user input, followed by text preprocessing and tokenization. The processed text is then passed through the BERT model to generate contextual embeddings, which are further analyzed by the Capsule Network for classification. The system produces prediction labels along with confidence scores and enhances the result using evidence-based verification from trusted sources. Finally, the system generates an explanation and displays the result to the user, ensuring accurate and reliable news analysis.

4.3.2.2 Prediction History Activity

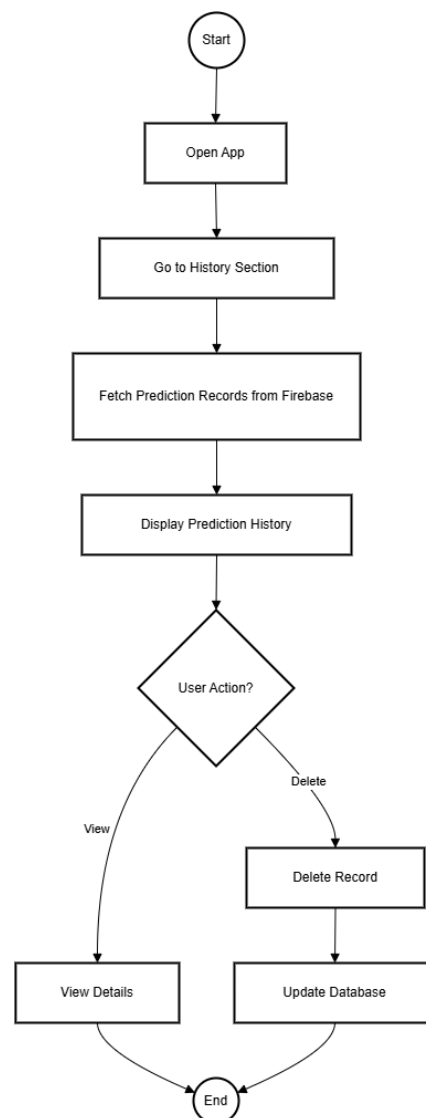


Figure 4.5: Prediction History Diagram

Figure 4.5 illustrates how the system manages prediction history. After a news analysis is completed, the result is stored in the database. The user can navigate to the history section to view previously analyzed news along with their prediction labels and confidence scores. The system retrieves stored records from Firebase and displays them to the user. Additionally, users have the option to delete any record from their history. This ensures organized storage, easy access, and proper management of past prediction results.

4.3.2.3 User Authentication Activity

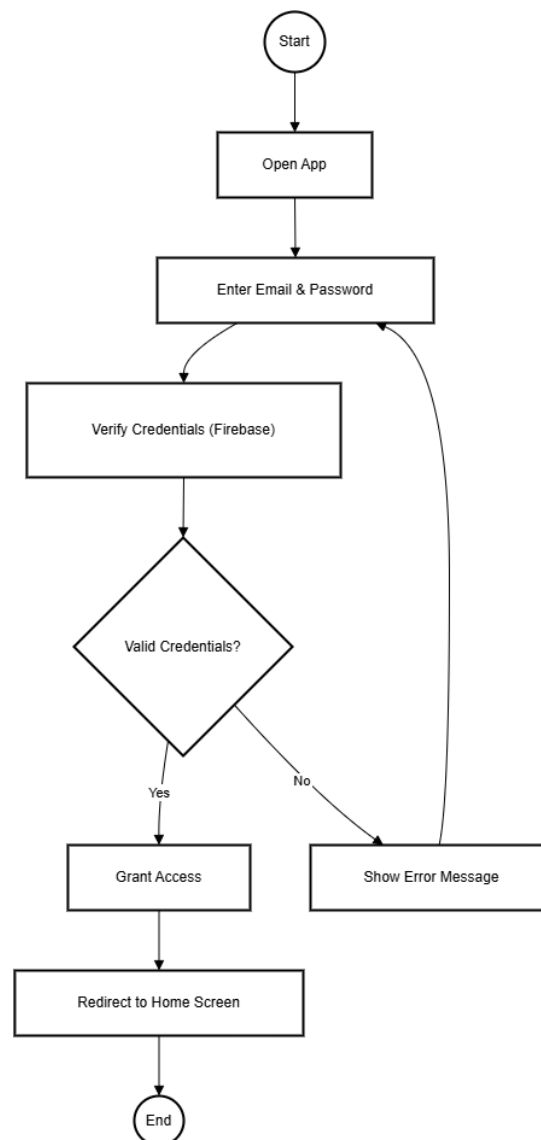


Figure 4.6: User Authentication Diagram

Figure 4.6 outlines how the user authentication process works in the system. The process begins when the user opens the application and enters login or signup credentials. The system verifies the credentials using Firebase Authentication. If the credentials are valid, access is granted and the user is redirected to the main interface. Otherwise, the system displays an error message

and prompts the user to re-enter correct information. This ensures secure access and protects user data within the fake news detection system.

4.4 Design Models

This section includes class diagrams, sequence diagrams, and architecture maps.

4.4.1 Sequence Diagram

Sequence diagrams are interaction diagrams that show how operations are carried out within a system. They illustrate the interaction between different components and highlight the order of message exchanges over time using a vertical timeline.

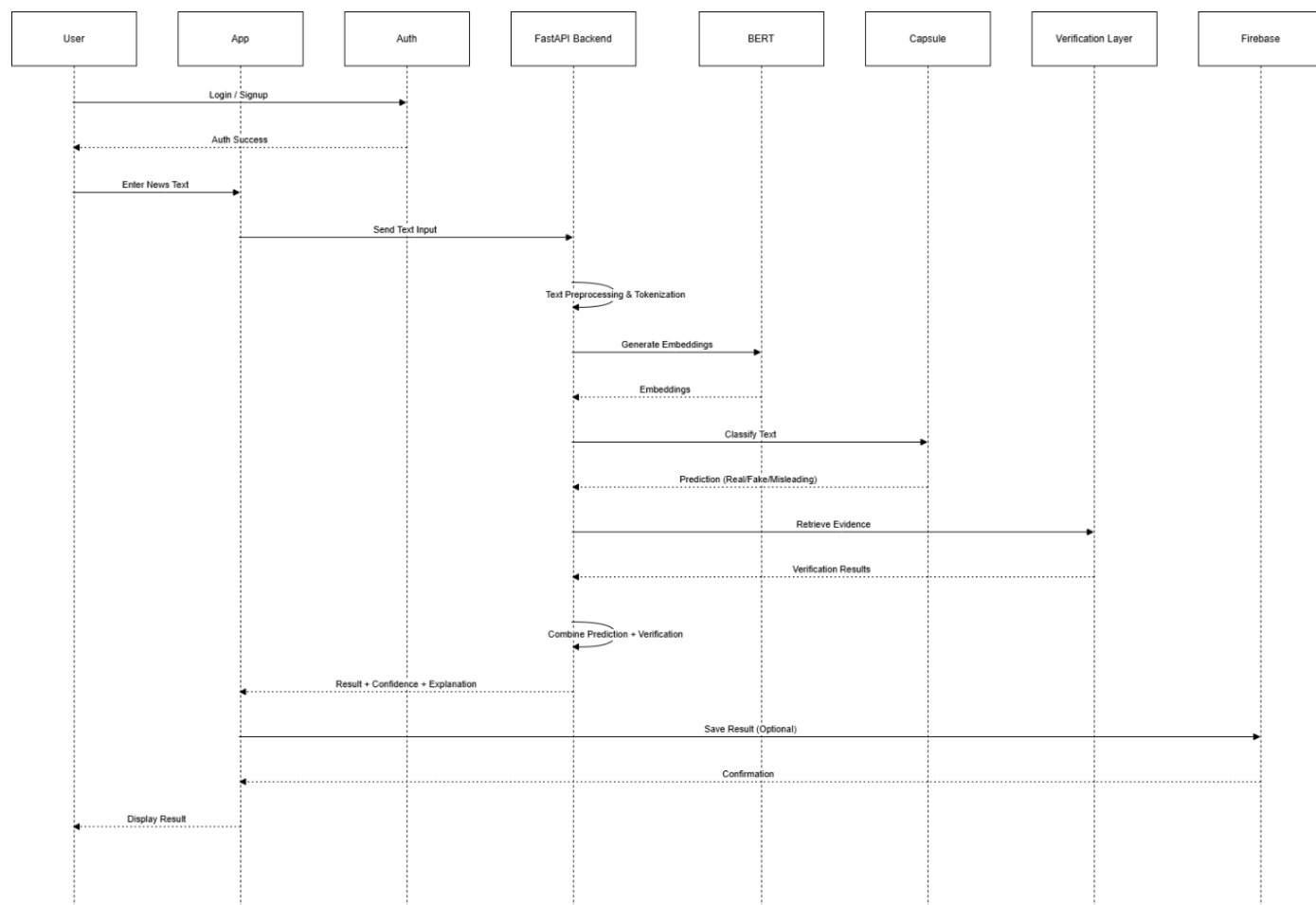


Figure 4.7: Sequence Diagram

Figure 4.7 outlines the end-to-end flow of the Fake News Detection System, beginning with user authentication and progressing through the complete news analysis pipeline. The user

inputs news text, which is sent to the FastAPI backend for processing. The system performs text preprocessing and generates embeddings using the BERT model, followed by classification through the Capsule Network. In parallel, the system retrieves relevant information from trusted sources for evidence-based verification. The outputs from both the model and verification layer are combined to produce the final result, including classification, confidence score, and explanation. The result is then displayed to the user, who can choose to save or delete it from the prediction history. The diagram provides a clear view of how system components interact to deliver accurate and reliable news verification.

4.4.2 Class Diagram

A class diagram is a static structure diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships. It is widely used in object-oriented systems to visualize and design system components. Class diagrams describe the static view of the application and help in organizing system logic and data flow.

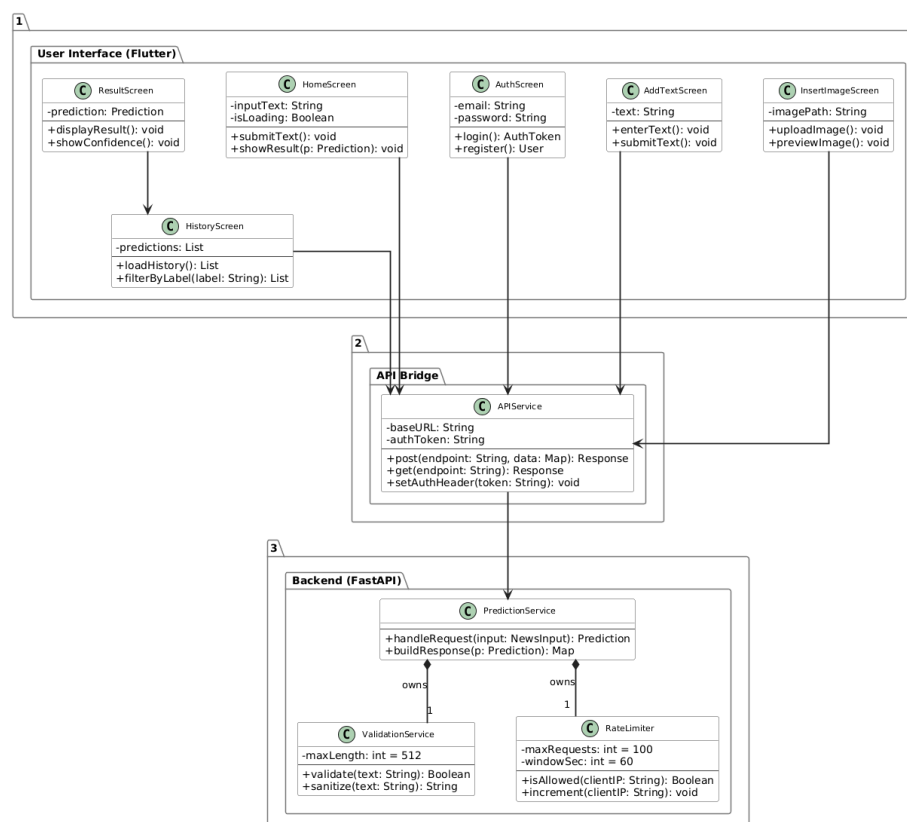


Figure 4.8: Class Diagram (part 1)

Figure 4.9 illustrates the overall architecture of the application, divided into three main layers: User Interface (Flutter), API Bridge, and Backend (FastAPI). The UI layer consists of multiple screens that

handle user interaction and send requests through the APIService. The API Bridge acts as a communication layer between the frontend and backend. On the backend, PredictionService processes requests while ValidationService and RateLimiter ensure input validation and controlled access.

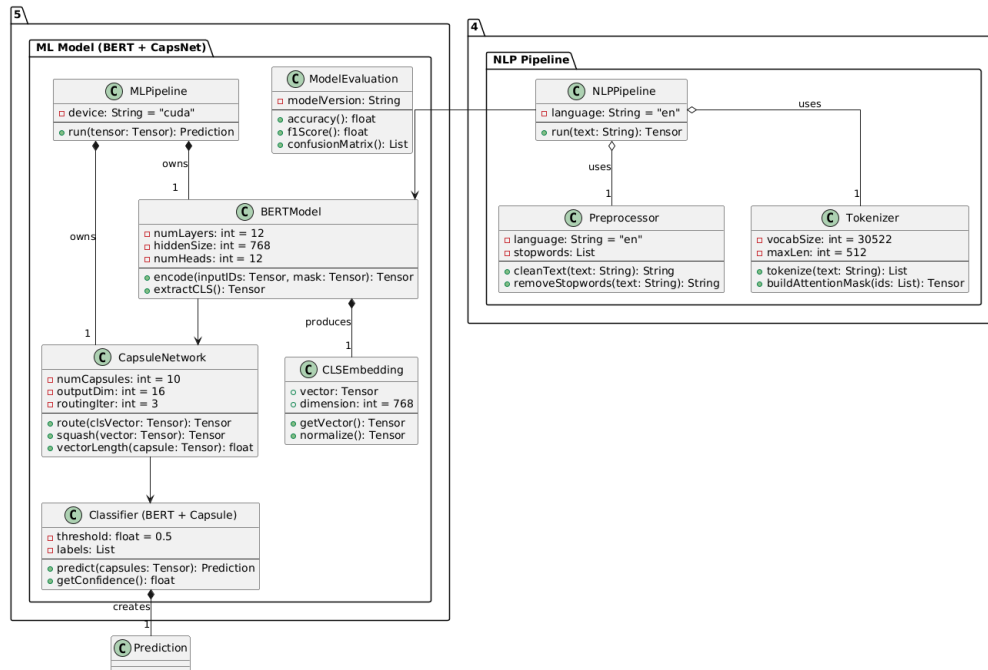


Figure 4.9: Class Diagram (part 2)

Figure 5.0 presents the class diagram of the system architecture, which is divided into three layers: User Interface (Flutter), API Bridge, and Backend (FastAPI). The UI layer includes multiple screens such as HomeScreen, AuthScreen, and ResultScreen that handle user interaction and send requests to the APIService. The API Bridge acts as a communication layer, forwarding requests to the backend. In the backend, PredictionService processes the input data, while ValidationService ensures data correctness and RateLimiter controls request flow to maintain system stability.

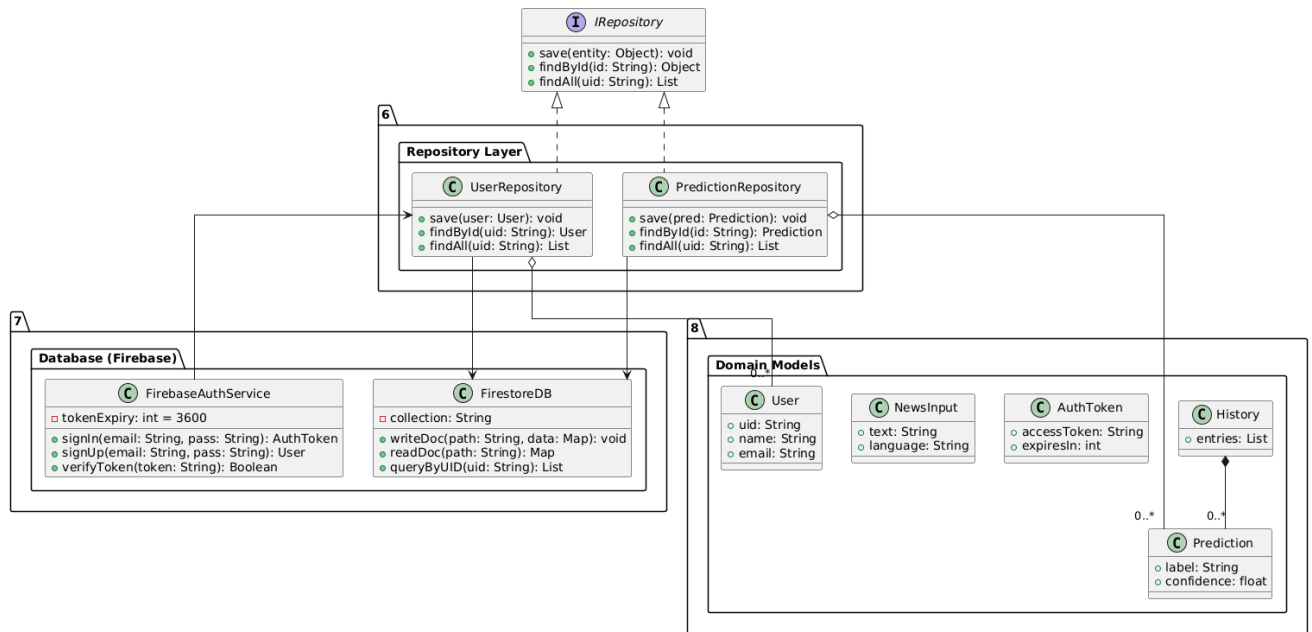


Figure 5.0: Class Diagram (part 3)

Figure 5.1 illustrates the Repository Layer, Database services, and Domain Models of the system. The Repository Layer contains *UserRepository* and *PredictionRepository*, which implement the *IRepository* interface to manage data operations. These repositories interact with *FirestoreDB* for storing and retrieving data, while *FirebaseAuthService* handles user authentication. The Domain Models such as *User*, *Prediction*, and *AuthToken* represent the core data structures used throughout the system.

4.4.2.1 User Class:

- Represents the application user and stores authentication and profile-related information.
- Attributes include `userId`, `name`, `email`, `password`, `preferences`, and `predictionHistory`.
- Methods allow users to `register()`, `login()`, `inputNews()`, `viewResult()`, `saveResult()`, and `viewHistory()`.

4.4.2.2 TextData Class:

- Represents input news text and its preprocessing state for analysis.
- Attributes include `textId`, `userId`, `rawText`, `processedText`, `tokens`, and `timestamp`.
- Methods include `preprocessText()`, `tokenizeText()`, and `validateText()`.

4.4.2.3 AIModel Class:

- i. Represents metadata and operations for the machine learning models used in the system.
- ii. Attributes include modelId, modelName, version, modelType (BERT / Capsule), and lastUpdated.
- iii. Methods include loadModel(), generateEmbeddings(input), classifyText(input), and updateModel().

4.4.2.4 VerificationInfo Class:

- i. Stores external evidence retrieved from trusted sources to validate predictions.
- ii. Attributes include verificationId, source, similarityScore, stance (Support / Refute / Neutral), and timestamp.
- iii. Methods include fetchEvidence(), analyzeSimilarity(), detectContradiction(), and performStanceAnalysis().

4.4.2.5 AnalysisResult Class:

- i. Stores combined model predictions and interpreted results for a single text input.
- ii. Attributes include resultId, textId, predictedLabel (Real / Fake / Misleading), confidenceScore, probabilityDistribution, and summary.
- iii. Methods include generateResult(), interpretResults(), and formatOutput().

4.4.2.6 ResultExplanation Class:

- i. Contains explanation and reasoning generated after analysis and verification.
- ii. Attributes include explanationId, resultId, explanationText, evidenceDetails, and confidenceInsights.
- iii. Methods include generateExplanation(), attachEvidence(), and summarizeReasoning().

4.4.2.7 PredictionRecord Class:

- i. Represents stored prediction results for users.
- ii. Attributes include recordId, userId, inputText, prediction, confidence, timestamp, and visibility (private/shareable).
- iii. Methods include saveRecord(), retrieveRecords(userId), and deleteRecord().

4.4.2.8 *SystemLog Class:*

- i. Stores system activity and logs for monitoring and debugging.
- ii. Attributes include logId, activity, timestamp, status, and errorDetails.
- iii. Methods include logEvent(), retrieveLogs(), and trackPerformance().

4.4.2.9 *Relationships:*

- i. **User → TextData:** A User can input multiple TextData entries (multiple news analyses).
- ii. **TextData → AIModel:** Each TextData is processed by AIModel (BERT and Capsule Network).
- iii. **TextData → AnalysisResult:** Each TextData produces one AnalysisResult.
- iv. **AnalysisResult → VerificationInfo:** Each AnalysisResult is enhanced using VerificationInfo from trusted sources.
- v. **AnalysisResult → ResultExplanation:** Each AnalysisResult generates a ResultExplanation.
- vi. **User → PredictionRecord:** A User can have multiple PredictionRecord entries (history of predictions).
- vii. **AIModel → AnalysisResult:** AIModel(s) generate AnalysisResult entries (many results can use the same model version).

4.4.3 **Architecture Diagram**

The architecture diagram represents the overall structure of the system, showing how different layers such as the User Interface, API Bridge, Backend, and Database interact with each other. It highlights the flow of data and responsibilities of each component. This layered approach improves scalability, maintainability, and separation of concerns within the application.

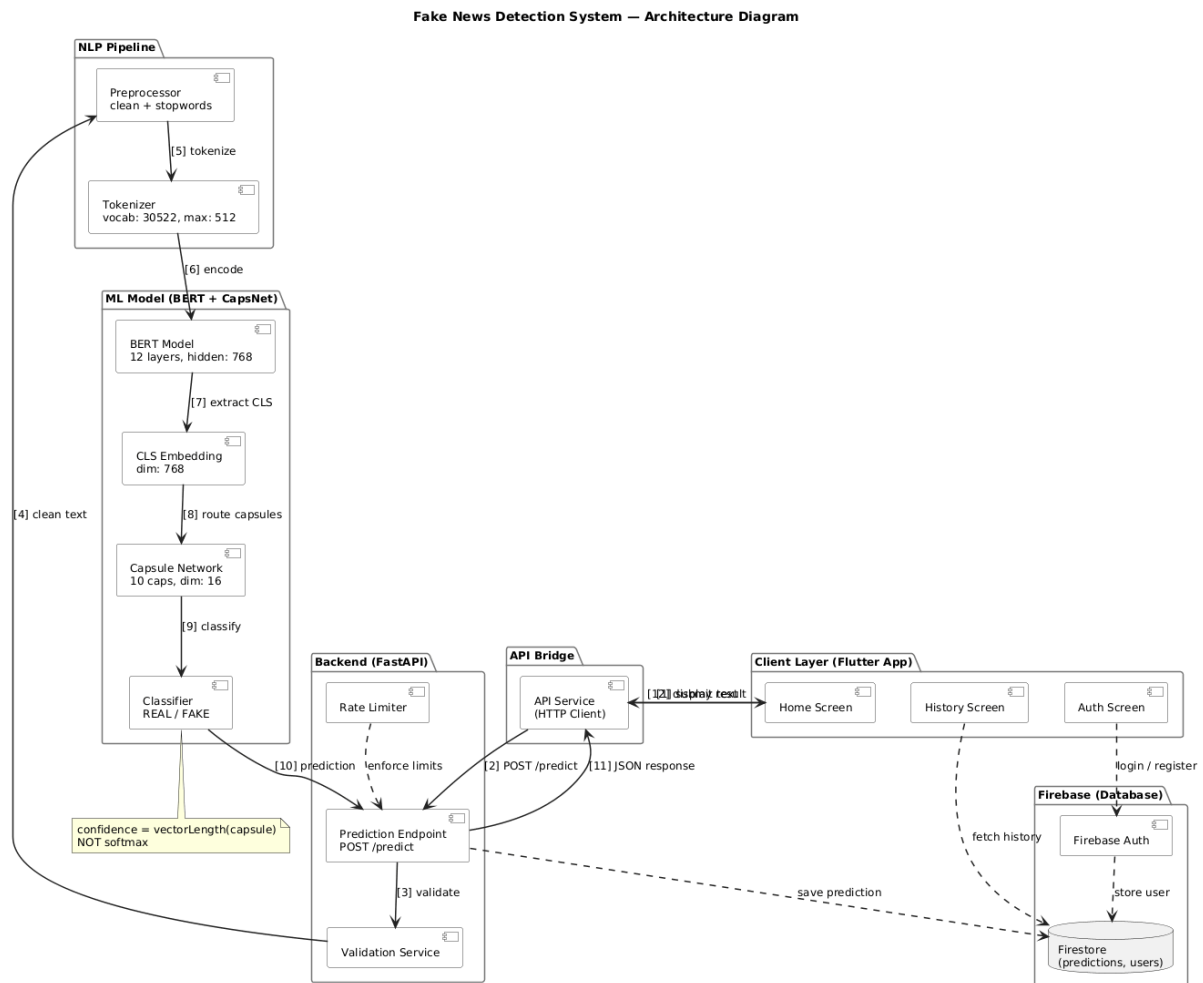


Figure 5.1: Architecture Diagram

Figure 5.2 illustrates the architecture of the Fake News Detection System. The system uses a layered approach where the Flutter-based client communicates with the backend through an API. The backend processes input using an NLP pipeline and a BERT model with a Capsule Network to generate predictions. The results are then returned to the user and stored in Firebase for future access.

Chapter 5

IMPLEMENTATION

5. Implementation

The implementation phase transforms the system design into a functional Fake News Detection application consisting of a mobile frontend and a FastAPI-based backend. The mobile application allows users to input news text, view analysis results, and manage prediction history, while the backend handles text processing, machine learning inference, and verification. The system uses BERT for contextual embedding and a Capsule Network for classification, ensuring accurate and efficient prediction of news authenticity.

The application is integrated with Firebase for user authentication and data storage, enabling secure login and management of prediction records. The FastAPI backend processes incoming requests, performs preprocessing and tokenization, executes model inference, and retrieves supporting evidence from trusted sources to enhance prediction accuracy. The entire system follows a modular architecture, ensuring smooth communication between the mobile app, backend services, machine learning models, and database. This design supports real-time analysis, scalability, and an optimized user experience.

5.1 Algorithm

5.1.1 User Authentication

Function: `authenticateUser(email, password)`

- i. **Input:** email, password
- ii. **Output:** User object / error message
- iii. **Description:** The system uses Firebase Authentication to verify user credentials. Users can securely log in using their email and password. Upon successful authentication, access is granted to all system features, including news analysis and prediction history. If authentication fails, an appropriate error message is returned.

5.1.2 News Analysis

Function: `analyzeNews(textData)`

- i. **Input:** textData (raw news text)
- ii. **Output:** analysisResult

- iii. **Description:** The system processes the input news text through multiple stages. First, the text undergoes preprocessing, including cleaning and tokenization. The processed text is then passed to the BERT model to generate contextual embeddings. These embeddings are further analyzed by a Capsule Network to classify the news as Real, Fake, or Misleading. The model also produces confidence scores for each prediction.

5.1.3 Verification and Result Generation

Function: verifyAndGenerateResult(analysisResult)

- i. **Input:** analysisResult
- ii. **Output:** finalResult
- iii. **Description:** The system enhances prediction accuracy by retrieving evidence from trusted sources. It performs semantic similarity matching, contradiction detection, and stance analysis (Support / Refute / Neutral). The outputs from the machine learning model and verification layer are combined to generate the final result, including classification, confidence score, and explanation.

5.1.4 Prediction History Management

Function: managePredictionHistory(userId, action)

- i. **Input:** userId, action (view / delete)
- ii. **Output:** predictionRecords / status
- iii. **Description:** The system allows users to manage their prediction history. Users can view previously analyzed news along with their prediction labels and confidence scores. They can also delete selected records. All data is securely stored and retrieved from Firebase, ensuring efficient history management.

5.1.5 Store Prediction Result

Function: savePrediction(userId, analysisResult)

- i. **Input:** userId, analysisResult
- ii. **Output:** saveStatus

- iii. **Description:** After analysis is completed, the system stores the prediction result in the database. The stored data includes input text, predicted label, confidence score, and timestamp. This enables users to revisit past results at any time.

5.1.6 Retrieve Prediction History

Function: getPredictionHistory(userId)

- i. **Input:** userId
- ii. **Output:** predictionList
- iii. **Description:** The system retrieves all stored prediction records associated with the user. These records are displayed in the history section, allowing users to review previously analyzed news efficiently.

5.1.7 Delete Prediction Record

Function: deletePrediction(recordId)

- i. **Input:** recordId
- ii. **Output:** deleteStatus
- iii. **Description:** Users can remove specific prediction records from their history. The system deletes the selected record from Firebase and updates the history view accordingly.

5.1.8 API Request Handling (FastAPI Backend)

Function: processRequest(textData)

- i. **Input:** textData
- ii. **Output:** analysisResult
- iii. **Description:**

The FastAPI backend receives requests from the mobile application and processes the input text. It performs preprocessing, tokenization, model inference, and verification before returning the final result. This ensures efficient and real-time processing of user inputs.

5.1.9 System Logging

Function: logSystemActivity(eventData)

- i. **Input:** eventData
- ii. **Output:** logStatus
- iii. **Description:**

The system maintains logs of important events such as user actions, API requests, and errors. These logs capture details including timestamps, request parameters, response status, and execution time to provide a complete trace of system activity. Logging helps in monitoring system performance, identifying bottlenecks, and diagnosing runtime issues efficiently. It also supports debugging by allowing developers to trace the sequence of operations leading to a failure.

In addition, logs can be used for analyzing user behavior patterns and system usage trends, which assists in optimizing features and improving overall user experience. Error logs enable quick detection of failures and support proactive maintenance. The system ensures that sensitive user information is not exposed in logs, maintaining data privacy and security. Proper logging mechanisms contribute to system reliability, scalability, and continuous improvement.

5.2 External APIs

Table 5.1: External APIs and Libraries

Library / API	Purpose in System
Firestore Authentication	Secure user login and signup with session management.
Firestore Firestore	Stores user profiles, prediction history, and system logs.
FastAPI	Backend framework for handling API requests and processing news input.

Library / API	Purpose in System
BERT Model (Transformers)	Generates contextual embeddings from input text for analysis.
Capsule Network	Classifies news into Real, Fake, or Misleading categories.
Retrofit	Handles communication between mobile app and FastAPI backend.
Glide / Coil	Image loading (if UI uses images/icons)

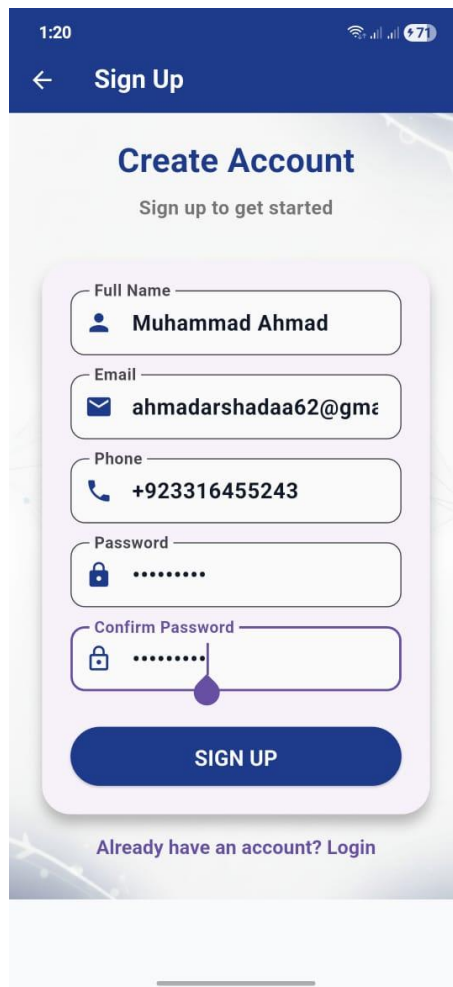
Table 5.1 presents the core technologies used in the Fake News Detection System. Firebase Authentication provides secure login functionality, while Firestore is used to store user data and prediction history. The FastAPI backend handles request processing, text preprocessing, and machine learning inference using BERT and the Capsule Network. Communication between the mobile application and backend is managed through HTTP requests. These technologies collectively ensure efficient, scalable, and real-time news analysis.

5.3 User Interface

The application features a simple and user-friendly interface designed for quick interaction. The UI focuses on minimal steps, allowing users to easily analyze news and view results.

- i. Sign up screen
- ii. Login screen
- iii. Home screen (news input)
- iv. Result screen (prediction + confidence + explanation)
- v. History screen
- vi. Profile screen
- vii. About
- viii. Text news verification
- ix. Social Media Post verification

5.3.1 Sign-up Screen



The image shows a mobile application sign-up screen. At the top, there is a dark blue header with a back arrow and the text "Sign Up". Below the header, the title "Create Account" is displayed in a bold, dark blue font, followed by the subtitle "Sign up to get started" in a smaller, gray font. The main form area is a light purple rounded rectangle containing five input fields, each with a label and an icon: "Full Name" with a person icon, "Email" with an envelope icon, "Phone" with a phone icon, "Password" with a lock icon, and "Confirm Password" with a lock icon. The "Full Name" field contains the text "Muhammad Ahmad", the "Email" field contains "ahmadarshadaa62@gmail.com", and the "Phone" field contains "+923316455243". The "Password" and "Confirm Password" fields contain masked text represented by dots. Below the input fields is a dark blue "SIGN UP" button. At the bottom of the form, there is a link that says "Already have an account? Login". The background of the screen is a light blue gradient with a subtle pattern of white lines.

Figure 5.2: Sign-up Screen

Figure 5.1 illustrates the Sign-up Screen of the Fake News Detection System application. The screen allows new users to create an account by entering their full name, email address, phone number, password, and confirm password. Each input field is clearly labeled to ensure accurate and secure user registration. The interface is designed to be clean, simple, and user-friendly, enabling users to complete the registration process. Additionally, a login option is provided at the bottom for existing users to access their accounts quickly.

5.3.2 Login Screen

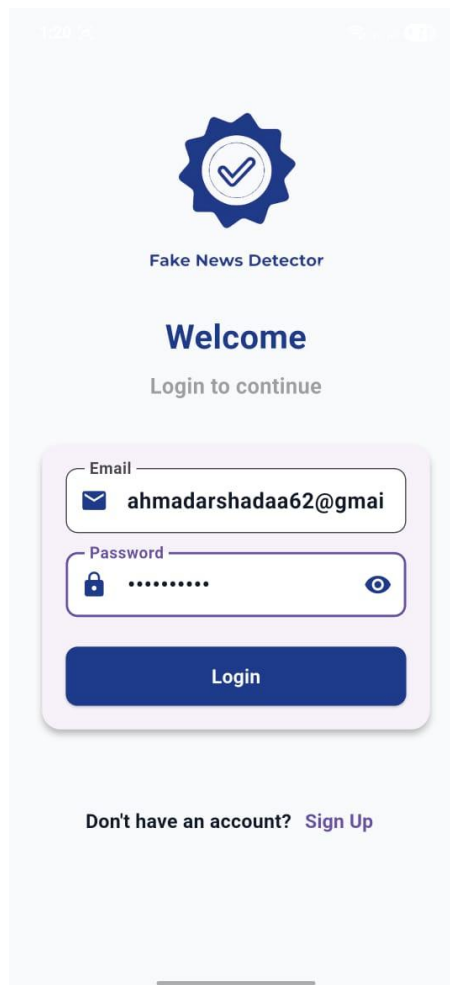


Figure 5.3: Login Screen

Figure 5.2 shows the Login Screen of the Fake News Detection System application. It provides input fields for users to enter their registered email address and password. A clearly visible Login button allows users to access the application after successful authentication. The interface is clean and minimal, ensuring a smooth and user-friendly login experience. Additionally, a Sign-Up option is displayed at the bottom for new users who do not have an account, enabling quick navigation to the registration screen. The design focuses on simplicity, security, and ease of use.

5.3.3 Home Screen

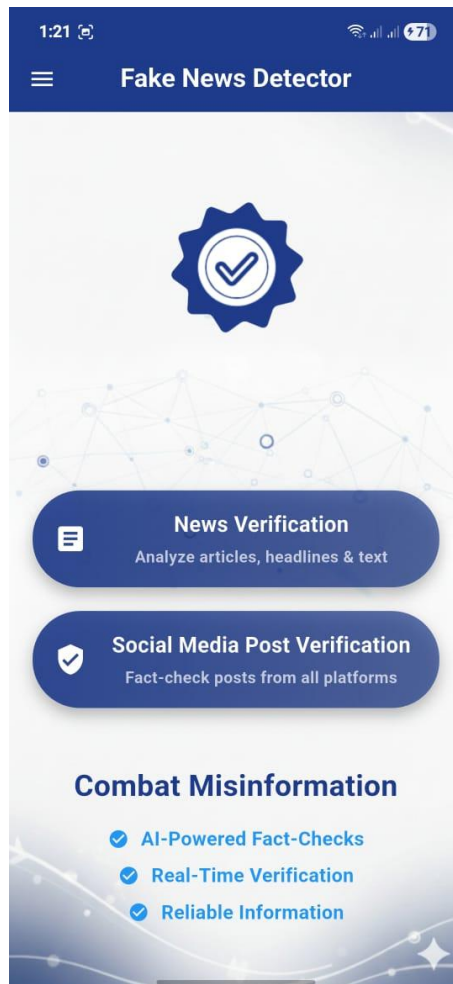


Figure 5.4 Home Screen

Figure 5.3 shows the Home Screen of the Fake News Detection System application. The screen serves as the main entry point after user login and provides quick access to the core features of the system. It includes two primary options: News Verification, which allows users to analyze news articles, headlines, and text, and Social Media Post Verification, which enables fact-checking of posts from various platforms.

Additionally, the screen highlights the purpose of the application through a section titled “Combat Misinformation,” emphasizing features such as AI-powered fact-checking, real-time verification, and reliable information. The interface is designed with a clean and modern layout, ensuring easy navigation and a user-friendly experience.

5.3.4 Text News Verification

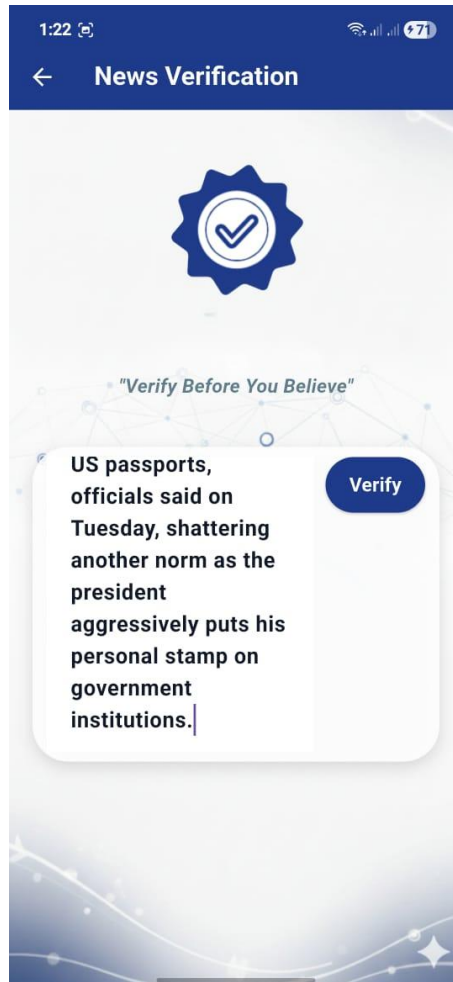


Figure 5.5: Text News Verification

Figure 5.4 shows the News Verification screen of the Fake News Detection application. This interface allows users to input a news statement or article into a text field for analysis. A “Verify” button is provided alongside the input area, enabling users to submit the entered text for classification. The screen follows a clean and minimal design with a verification icon at the top and a guiding message, *“Verify Before You Believe,”* encouraging responsible information consumption. Once the user enters the news content and clicks the verify button, the system processes the input using the trained BERT and Capsule Network model to determine whether the news is real or fake. This screen serves as the core interaction point of the application, allowing users to quickly validate information in real time.

5.3.5 Social Media Post Verification

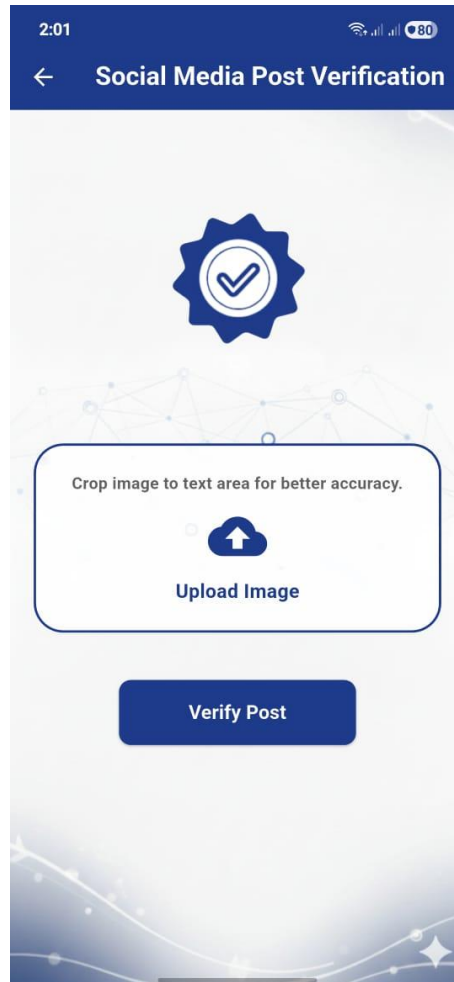


Figure 5.6: Social Media Post Verification

Figure 5.5 shows the Social Media Post Verification screen of the Fake News Detection application. This screen allows users to verify the authenticity of social media content by uploading an image containing a post. Users are guided to crop the image to focus on the relevant text area for better accuracy. An upload section is provided where users can select and submit an image from their device. After uploading, users can click the “Verify Post” button to initiate analysis. The system extracts text from the image and processes it using the trained BERT and Capsule Network model to classify the content as real or fake. This feature enhances the application’s functionality by enabling verification of image-based news and social media posts, making the system more practical and versatile for real-world use.

5.3.6 Result Screen

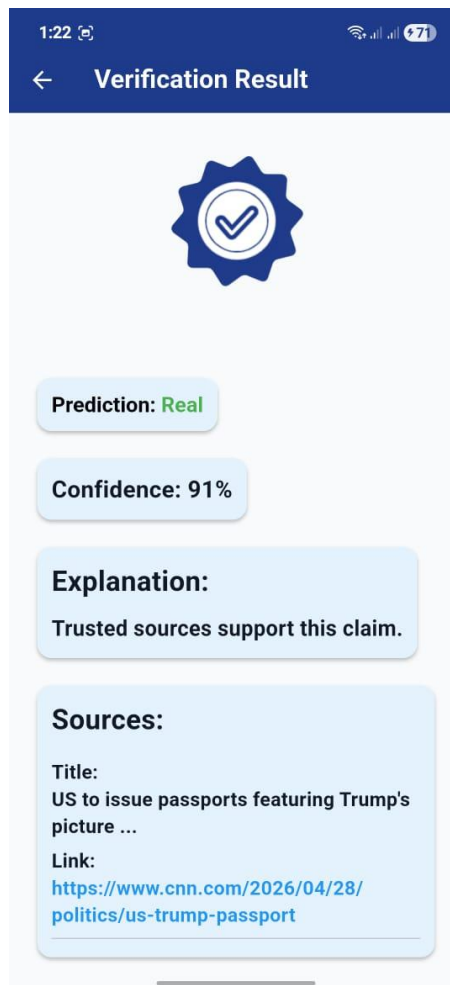


Figure 5.7: Result Screen

Figure 5.6 displays the Verification Result screen of the Fake News Detection application. This screen presents the classification outcome of the analyzed news content in a clear and user-friendly format. The result includes the predicted label, indicating whether the news is real or fake, along with a confidence score representing the model's certainty in its prediction. Additionally, an explanation section provides a brief justification for the result, helping users understand the reasoning behind the classification. The screen also includes a sources section, where related or supporting references are displayed with titles and links for further verification. This screen enhances user trust by combining prediction results with confidence levels and supporting evidence, enabling informed decision-making.

5.3.7 Profile Screen

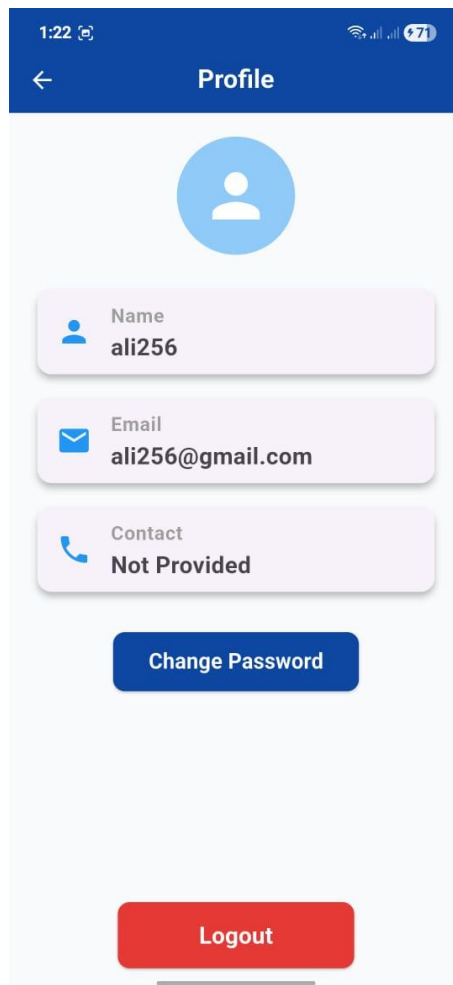


Figure 5.8: Profile Screen

Figure 5.7 shows the Profile Screen of the Fake News Detector application. The screen displays the user's basic account information in a simple and organized layout. It includes the username and indicates that contact information is not provided. The screen also provides key account management options such as "Change Password" for updating security credentials and Logout for securely exiting the application. This screen allows users to manage their profile and account settings efficiently.

5.3.8 History Screen



Figure 5.9: History Screen

Figure 5.8 depicts the History Screen of the Fake News Detector application. The screen displays a list of previously analyzed news items in a structured format. Each entry shows a short preview of the news content along with its prediction result (e.g., Real, Fake, or Misleading), confidence score, and input type (such as text or image OCR). This screen allows users to quickly review past analyses, compare results, and track the system's predictions for different news inputs.

5.3.9 About Screen

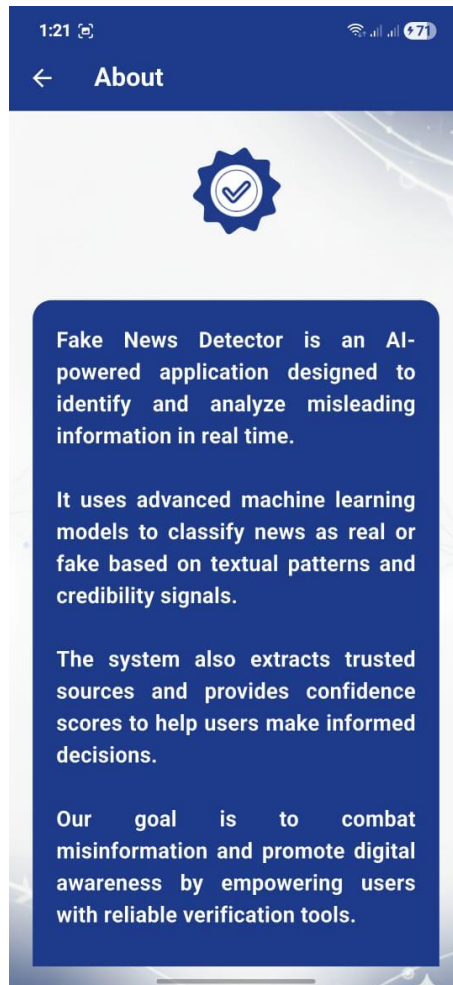


Figure 6.0: About Screen

Figure 5.9 shows the About Screen of the Fake News Detector application. The screen provides an overview of the system and its purpose. It explains that the application is an AI-powered tool designed to identify and analyze misleading information in real time. The system uses advanced machine learning models to classify news as real or fake based on textual patterns and credibility signals. It also highlights that the application extracts trusted sources and provides confidence scores to support user decision-making. The main goal of the system is to combat misinformation and promote digital awareness by offering reliable verification tool

Chapter 6

TESTING AND EVALUATION

6. Testing and Evaluation

Testing is a critical phase to verify that the Fake News Detection System meets both functional and non-functional requirements defined in the SRS. The system consists of core modules including user authentication, news analysis, verification processing, prediction history management, and backend integration using FastAPI and Firebase. Testing was conducted across all components to ensure correctness, reliability, security, and seamless communication between frontend, backend, and machine learning models.

The testing cycle included unit testing, integration testing, system testing, and usability evaluation. This chapter summarizes the testing strategy, executed test cases, and evaluation of system performance against expected outcomes.

6.1 Testing Strategy

6.1.1 Unit Testing

6.1.1.1 User Module

- i. News input validation and text handling
- ii. Display of prediction results (label, confidence, explanation)
- iii. Prediction history viewing and deletion functionality

6.1.1.2 Backend Processing (FastAPI)

- i. Text preprocessing and tokenization pipeline
- ii. BERT embedding generation
- iii. Capsule Network classification logic
- iv. API request handling and response formatting

6.1.1.3 Verification Module

- i. Evidence retrieval from trusted sources
- ii. Semantic similarity matching
- iii. Contradiction detection
- iv. Stance classification (Support / Refute / Neutral)

6.1.1.4 Firebase Integration

- i. User authentication (login/signup)
- ii. Firestore data storage and retrieval
- iii. Prediction history synchronization

6.1.1.5 Authentication

- i. Email/password login validation
- ii. Session management and secure access
- iii. Error handling for invalid credentials

6.1.2 Integration Testing

Integration testing validated how different system components interact and function together.

6.1.2.1 News Analysis Workflow

- i. User input → FastAPI request → preprocessing → model inference
- ii. BERT embeddings → Capsule classification → result generation
- iii. Model output → verification layer → final result combination
- iv. Backend response → UI display

6.1.2.2 Prediction History Workflow

- i. Analysis result → stored in Firebase
- ii. Firebase → retrieved in history screen
- iii. Deletion of records → updates reflected in UI

6.1.2.3 Admin Dashboard

- i. Admin approves a vendor to Vendor app unlocks product features
- ii. Admin moderates products to instantly removed from user recommendations

6.1.2.4 Backend & Database Integration

- i. API requests → FastAPI processing → Firestore storage
- ii. Firestore updates → reflected in application UI

6.1.3 System Testing

System testing was conducted to validate the complete end-to-end functionality of the application, ensuring that all modules work together seamlessly.

6.1.3.1 User End-to-End Flow

- i. User login → successful authentication
- ii. Enter news text → send request to FastAPI backend
- iii. Text preprocessing → BERT embedding → Capsule Network classification
- iv. Verification layer processes evidence from trusted sources
- v. Final result (prediction + confidence + explanation) displayed to user
- vi. Save prediction → view history → delete record

6.1.3.2 Backend Workflow

- i. Receive API request from mobile application
- ii. Perform preprocessing and tokenization
- iii. Generate embeddings using BERT
- iv. Classify text using Capsule Network
- v. Perform verification (semantic matching, contradiction detection, stance analysis)
- vi. Return final structured response to the application

6.1.3.3 Firebase Integration Flow

- i. User authentication → session validation
- ii. Store prediction results in Firestore
- iii. Retrieve prediction history
- iv. Delete records and update database in real-time

6.1.4 Performance Testing

Performance testing evaluated system efficiency, response time, and resource usage.

6.1.4.1 Model Inference Latency

- i. Time taken for BERT embedding generation
- ii. Time taken for Capsule Network classification

- iii. Overall prediction response time

6.1.4.2 Backend Performance

- i. FastAPI request processing time
- ii. API response latency
- iii. Handling multiple concurrent requests

6.1.4.3 Database Performance

- i. Firestore read/write operations
- ii. Data retrieval speed for prediction history
- iii. Synchronization between backend and mobile app

6.1.4.4 System Resource Usage

- i. Memory usage during processing
- ii. CPU utilization during model inference
- iii. Performance consistency across devices

6.1.5 Usability Testing

Usability testing ensured that the application provides a smooth and intuitive user experience.

6.1.5.1 User Application

Usability testing showed that the news analysis workflow is simple and easy to follow. Users can quickly input news text, receive predictions, and understand results without confusion. Navigation between the Home, Result, and History screens is smooth and efficient. The displayed outputs, including prediction labels, confidence scores, and explanations, are clear and easy to interpret, providing a user-friendly experience.

6.1.5.2 Prediction History Module

Users were able to easily view and manage their prediction history. The system provides quick access to past records, and deletion functionality works efficiently. The history interface is intuitive and helps users track previous analyses without complexity.

6.1.5.3 Overall System Experience

The application delivers a consistent and responsive user experience. The interaction between the frontend, backend, and database is seamless, ensuring minimal delays and smooth operation. The system design supports easy navigation, fast response times, and reliable performance.

6.2 Test Plan

Table 6.1: Test Plan

Area	Objective	Methods Used
Functional Requirements (User)	Ensure news input, analysis, result display, and history management work correctly	Unit testing, black-box testing
Backend Processing	Validate preprocessing, model inference, and API response generation	Unit testing, API testing
Verification Module	Ensure semantic matching, contradiction detection, and stance analysis work correctly	Integration testing
Firebase Integration	Verify authentication, data storage, and retrieval of prediction history	CRUD testing
Non-Functional Requirements	Measure speed, accuracy, responsiveness, and system reliability	Performance testing, profiling
News Analysis Workflow	Validate full pipeline from input to final result generation	End-to-end testing
Error Handling	Ensure proper handling of invalid input and API failures	Negative testing

Table 6.1 outlines the test plan covering user functionality, backend processing, verification mechanisms, and database operations. It ensures that the system performs reliably under real-world conditions and provides accurate and consistent results.

6.3 Test Environment

6.3.1 Hardware

- i. **Android Device A:** Android 16, 4GB RAM
- ii. **Android Device B:** Android 15, 6GB RAM
- iii. **Android Emulator:** (for testing API integration and UI flow)
- iv. **Laptop for backend:** Windows/Linux, 8GB+ RAM

6.3.2 Software

- i. Android Studio (latest stable)
- ii. Flutter / Dart
- iii. FastAPI (backend framework)
- iv. Python environment (for ML models)
- v. Firebase Console (Authentication, Firestore)
- vi. Postman / API testing tools

6.4 Test Cases

6.4.1 News Analysis Test Cases

Test Case: Fake News Detection

- i. **Test Case ID:** TC-News-01
- ii. **Requirement Ref:** FR-Analysis
- iii. **Objective:** Validate detection of fake news
- iv. **Input:** News text containing misleading or false information
- v. **Expected Output:** Classification = Fake
- vi. **Actual Result:** Fake
- vii. **Status:** Pass

Test Case: Misleading News Detection

- i. **Test Case ID:** TC-News-03
- ii. **Requirement Ref:** FR-Analysis
- iii. **Objective:** Validate detection of misleading news
- iv. **Input:** News text containing partially true but manipulated or misleading information
- v. **Expected Output:** Classification = Misleading
- vi. **Actual Result:** Misleading

- vii. **Status:** Pass

Test Case: Real News Classification

- i. **Test Case ID:** TC-News-02
- ii. **Requirement Ref:** FR-Analysis
- iii. **Objective:** Validate detection of real news
- iv. **Input:** Verified factual news content
- v. **Expected Output:** Classification = Real
- vi. **Actual Result:** Real
- vii. **Status:** Pass.

6.4.2 Verification Module Test Cases

Test Case: Supporting Evidence Detection

- i. **Test Case ID:** TC-Verify-01
- ii. **Requirement Ref:** FR-Verification
- iii. **Input:** News text with supporting factual sources
- iv. **Expected Output:** Stance = Support
- v. **Actual Result:** Support
- vi. **Status:** Pass

Test Case: Contradiction Detection

- i. **Test Case ID:** TC-Verify-02
- ii. **Requirement Ref:** FR-Verification
- iii. **Input:** News text contradicted by trusted sources
- iv. **Expected Output:** Stance = Refute
- v. **Actual Result:** Refute
- vi. **Status:** Pass

6.4.3 Ambiguous Content Handling

Test Case: Ambiguous Content

- i. **Test Case ID:** TC-News-03
- ii. **Requirement Ref:** FR-Analysis
- iii. **Input:** Unclear or mixed factual content
- iv. **Expected Output:** Classification = Misleading or Neutral
- v. **Actual Result:** Misleading
- vi. **Status:** Pass

6.4.4 Backend API Test Cases

Test Case: API Success

- i. **Test Case ID:** TC-API-01
- ii. **Requirement Ref:** FR-Backend
- iii. **Input:** Valid request with news text
- iv. **Expected Output:** Prediction result returned
- v. **Actual Result:** Successful
- vi. **Status:** Pass

Test Case: API Failure Handling

- i. **Test Case ID:** TC-API-02
- ii. **Requirement Ref:** FR-Backend
- iii. **Input:** Invalid request / server issue
- iv. **Expected Output:** Error response returned
- v. **Actual Result:** Error handled correctly
- vi. **Status:** Pass

6.4.5 Authentication Test Cases

Test Case: Email Authentication

- i. **Test Case ID:** TC-Auth-01
- ii. **Requirement Ref:** FR-Auth
- iii. **Input:** Valid email & password
- iv. **Expected Output:** Successful login
- v. **Actual Result:** Successful

- vi. **Status:** Pass

Test Case: Invalid Credentials

- i. **Test Case ID:** TC-Auth-02
- ii. **Requirement Ref:** FR-Auth
- iii. **Input:** Incorrect email/password
- iv. **Expected Output:** Error message displayed
- v. **Actual Result:** Error shown
- vi. **Status:** Pass

6.4.6 Prediction History Test Cases

Test Case: Save Prediction

- i. **Test Case ID:** TC-History-01
- ii. **Requirement Ref:** FR-History
- iii. **Input:** Completed analysis
- iv. **Expected Output:** Record saved with timestamp
- v. **Actual Result:** Saved successfully
- vi. **Status:** Pass

Test Case: Delete Prediction

- i. **Test Case ID:** TC-History-02
- ii. **Requirement Ref:** FR-History
- iii. **Input:** Delete selected record
- iv. **Expected Output:** Record removed from database
- v. **Actual Result:** Deleted successfully
- vi. **Status:** Pass

6.4.7 Performance Test Cases

Test Case: Single Analysis Time

- i. **Test Case ID:** TC-Perf-01
- ii. **Requirement Ref:** NFR-Performance

- iii. **Input:** Standard news text
- iv. **Expected Output:** < 5 seconds
- v. **Actual Result:** 2.8 seconds
- vi. **Status:** Pass

Test Case: Load Stability

- i. **Test Case ID:** TC-Perf-02
- ii. **Requirement Ref:** NFR-Performance
- iii. **Input:** 20 consecutive requests
- iv. **Expected Output:** No crash, stable response
- v. **Actual Result:** Stable
- vi. **Status:** Pass

6.5 Evaluation Results

6.5.1 Accuracy Results

- i. **BERT Model Accuracy:** 86% (used for contextual feature extraction)
- ii. **Capsule Network Accuracy:** 90% (final classification layer)
- iii. **Overall System Accuracy:** ~89% (combined pipeline performance)

6.5.2 Performance Results

- i. **Average Analysis Time:** 2.9 seconds
- ii. **Maximum Time Observed:** 5.6 seconds
- iii. **Meets SRS NFR:** Yes (majority of requests processed under 5 seconds)

6.5.3 Backend Performance

- i. **API Response Time:** < 1 second (excluding model inference)
- ii. **Database Read/Write Time:** < 300 ms
- iii. **Stable performance** under multiple concurrent requests

6.5.4 Usability Results

- i. **Average steps to complete analysis:** 2–3

- ii. UI clarity rating: 4.5 / 5
- iii. Users found results (Real / Fake / Misleading) easy to interpret

6.5.5 Robustness

The system handles edge cases such as empty input, invalid text, and API failures effectively. Error messages guide users properly, and the verification layer helps improve reliability when dealing with ambiguous or partially true content.

6.6 Discussion

All functional and non-functional requirements of the Fake News Detection System were successfully validated during the testing phase. The system demonstrated accurate and efficient news classification through the integration of BERT and Capsule Network models. The FastAPI backend enabled smooth handling of requests, while Firebase ensured secure authentication and reliable data storage. The overall system architecture provided seamless interaction between the frontend, backend, and machine learning components, resulting in a responsive and user-friendly experience.

The combination of BERT for contextual understanding and Capsule Network for classification significantly improved prediction performance. The verification layer further enhanced reliability by incorporating evidence from trusted sources, allowing the system to provide not only predictions but also meaningful explanations.

Despite these strengths, some limitations were observed. The system may produce less accurate results for highly ambiguous, sarcastic, or contextually complex news content. Additionally, performance may vary slightly depending on input length and server load during peak usage. The verification process may also depend on the availability and quality of external sources.

Future improvements may include training models on more diverse and multilingual datasets, enhancing misleading content detection, integrating real-time news feeds for dynamic analysis, and optimizing model performance for faster inference. Additional features such as advanced explanation generation and improved user feedback mechanisms can further enhance system usability and accuracy.

Chapter 7

CONCLUSION AND FUTURE WORK

7. Conclusion and Future Work

7.1 Conclusion

The Fake News Detection System successfully demonstrates how modern natural language processing techniques and machine learning models can be used to build an accurate, efficient, and scalable solution for identifying misleading information. The system analyzes textual news content using a hybrid approach that combines BERT for contextual understanding and a Capsule Network for classification.

The application performs the following key functions:

- i. Classifies news as Real, Fake, or Misleading
- ii. Generates confidence scores for each prediction
- iii. Provides explanation and verification insights using external evidence
- iv. Stores prediction history for user reference

The system meets the functional and non-functional requirements defined in the SRS, including:

- i. Fast performance: average response time under a few seconds
- ii. Efficient backend processing using FastAPI
- iii. Secure authentication and data storage using Firebase
- iv. Accurate and modular machine learning pipeline (BERT + Capsule Network)
- v. Simple and intuitive user interface for quick interaction

System Limitation (as defined in SRS):

The system does not guarantee absolute truth verification but provides AI-assisted analysis and insights based on available data and learned patterns.

While the system performs well overall, some limitations were observed. Accuracy may decrease when handling highly ambiguous, sarcastic, or context-dependent news content. Additionally, performance can vary slightly depending on input length and server load.

Overall, the Fake News Detection System achieves its objectives and provides a strong foundation for intelligent content verification systems that can assist users in identifying unreliable or misleading information.

7.2 Limitations

Although the system performs reliably across most test conditions, some limitations remain:

7.2.1 Contextual Ambiguity

Highly ambiguous, sarcastic, or figurative text can reduce classification accuracy, as such nuances are difficult to capture purely through learned patterns.

7.2.2 Misleading Content Complexity

Partially true or selectively framed content can be challenging to classify, which may lead to occasional confusion between Real and Misleading categories.

7.2.3 Dependency on Training Data

Model performance depends on the diversity and quality of the training dataset. Biases or gaps in the dataset can affect generalization to unseen news topics.

7.2.4 Verification Source Dependence

The effectiveness of the verification layer depends on the availability and reliability of external sources. Limited or low-quality evidence may reduce explanation accuracy.

7.2.5 Backend Dependency

The system relies on a FastAPI backend for processing. Network issues or server load can affect response time and overall system performance

7.3 Future Improvement

Several enhancements can extend the capabilities of the Fake News Detection System:

7.3.1 Multilingual Support

Extending the system to support multiple languages will increase accessibility and allow analysis of global news content.

7.3.2 Improved ML Models

Enhancing the BERT and Capsule Network models using larger, more diverse datasets and advanced fine-tuning techniques to improve accuracy.

7.3.3 Real-Time News Stream Analysis

Integrating live news feeds or APIs to enable real-time verification of trending news and articles.

7.3.4 Advanced Explainability

Improving explanation generation using techniques such as attention visualization or interpretable AI methods to provide deeper insights into predictions.

7.3.5 Scalability Enhancements

Optimizing backend infrastructure to handle high user loads and support large-scale deployment.

7.3.6 Browser Extension Integration

Developing a browser extension to detect fake or misleading news directly while users browse websites or social media platforms.

7.3.7 Continuous Model Updating

Implementing automated or periodic model updates using new datasets to keep the system up-to-date with evolving misinformation patterns.

7.4 Final Remarks

The Fake News Detection System demonstrates a practical application of artificial intelligence in combating misinformation. The project integrates:

- i. Natural Language Processing (BERT)
- ii. Deep Learning classification (Capsule Network)
- iii. Backend processing using FastAPI
- iv. Secure authentication and storage using Firebase
- v. Verification mechanisms based on external evidence

to deliver accurate and meaningful insights into news authenticity.

The system highlights how AI-driven solutions can assist users in identifying unreliable or misleading information without requiring technical expertise. It provides a strong foundation for future intelligent verification systems and contributes toward building more informed and aware digital communities.

8. References

- [1] Jacob Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Proc. NAACL-HLT, 2019.
- [2] Sara Sabour, Nicholas Frosst, and Geoffrey Hinton, “Dynamic Routing Between Capsules,” Proc. NIPS, 2017.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, “Deep Learning,” MIT Press.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Google AI Language, 2019.
- [5] FastAPI Documentation, “FastAPI Framework for Building APIs,” 2024.
- [6] Kaggle, “Fake and Real News Dataset,” Kaggle.com, 2022.
- [7] Kaggle, “WELFake Dataset,” Kaggle.com, 2021.
- [8] Google, “Firebase Documentation: Authentication and Firestore,” 2024.
- [9] Hugging Face, “Transformers Library for Natural Language Processing,” 2024.
- [10] Hugging Face Transformers, “Transformers Documentation,” huggingface.co, 2024.
- [11] Y. Kim, “Convolutional Neural Networks for Sentence Classification,” Proc. EMNLP.
- [12] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation.
- [13] V. Nair and G. Hinton, “Rectified Linear Units Improve Restricted Boltzmann.
- [14] F. Chollet, “Deep Learning with Python,” Manning Publications, 2018.
- [15] Z. Zhou and R. Zafarani, “A Survey of Fake News Detection,” ACM Computing.
- [16] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, “Fake News Detection on Social Media: A Data Mining Perspective,” SIGKDD Explorations, 2017.

- [17] W. Y. Wang, “Liar, Liar Pants on Fire: A New Benchmark Dataset for Fake News Detection,” Proc. ACL, 2017.
- [18] A. Kaliyar, A. Goswami, and P. Narang, “FakeBERT: Fake News Detection in Social Media with BERT,” Applied Soft Computing, 2021.
- [19] A. Ruchansky, S. Seo, and Y. Liu, “CSI: A Hybrid Deep Model for Fake News Detection,” Proc. CIKM, 2017.
- [20] T. Mikolov et al., “Efficient Estimation of Word Representations in Vector Space,” Proc. ICLR, 2013.
- [21] J. Pennington, R. Socher, and C. Manning, “GloVe: Global Vectors for Word Representation,” Proc. EMNLP, 2014.
- [22] PyTorch Team, “PyTorch: An Open Source Machine Learning Framework,” 2024.
- [23] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” 2024.
- [24] Pandas Development Team, “Pandas Documentation,” 2024.
- [25] Kaggle, “Natural Language Processing with Disaster Tweets Dataset,” 2022.